

PÓS-GRADUAÇÃO EM CIÊNCIA DE DADOS
Centro Universitário de Brasília — CEUB

GUIA PRÁTICO DE
ANÁLISE DE DADOS E MACHINE
LEARNING
COM PYTHON E CRISP-DM

Apostila de Consulta — Scripts Genéricos e Referência Rápida
Introdução em Aprendizagem de Máquina · Aulas 1 e 2

Este material serve como guia de consulta permanente, aplicável a qualquer dataset.
Todos os scripts são genéricos e acompanhados de comentários explicativos.

O QUE É O CRISP-DM E POR QUE USÁ-LO?

O CRISP-DM (Cross-Industry Standard Process for Data Mining) é a metodologia mais utilizada em projetos de Ciência de Dados — adotada em mais de 70% dos projetos ao redor do mundo. Ele divide o trabalho em 6 fases iterativas e bem definidas, garantindo que o trabalho seja organizado, reproduzível e alinhado ao objetivo de negócio.

Fase	Nome	Objetivo Principal
1	Entendimento do Negócio	Definir o problema e os critérios de sucesso
2	Entendimento dos Dados	Explorar e conhecer os dados disponíveis (EDA)
3	Preparação dos Dados	Limpar, tratar e transformar os dados
4	Modelagem	Treinar e ajustar algoritmos de Machine Learning
5	Avaliação	Medir o desempenho real do modelo (RMSE, R^2 , MAE)
6	Implantação	Colocar o modelo em produção e monitorar



O processo é cíclico!

As fases não são lineares. É comum voltar à fase anterior ao descobrir algo novo.

Exemplo: durante a Modelagem (fase 4), você percebe que precisa criar uma nova variável → volta para Preparação (fase 3).

FASE 0 — CONFIGURAÇÃO DO AMBIENTE

Antes de iniciar qualquer projeto, execute a célula de configuração abaixo. Ela importa todas as bibliotecas necessárias e define padrões visuais.

0.1 — Importações e Configurações Globais

```
# — IMPORTAÇÕES ESSENCIAIS —————
# pandas: manipulação de tabelas (DataFrames)
import pandas as pd

# numpy: cálculos matemáticos e arrays
import numpy as np

# matplotlib: criação de gráficos (biblioteca base)
import matplotlib.pyplot as plt

# seaborn: gráficos estatísticos com visual aprimorado
import seaborn as sns

# scikit-learn: algoritmos de Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# pickle: salvar e carregar modelos treinados
import pickle

# — CONFIGURAÇÕES VISUAIS —————
# permite ver gráficos diretamente no Jupyter/Colab
%matplotlib inline

# tamanho padrão dos gráficos (largura x altura em polegadas)
plt.rcParams['figure.figsize'] = (12, 5)

# tema visual do seaborn (opções: whitegrid, darkgrid, white, dark)
sns.set_theme(style='whitegrid')

# formatar números com 2 casas decimais nas tabelas
pd.set_option('display.float_format', '{:.2f}'.format)

# — VERIFICAÇÃO —————
print(f"Pandas:      {pd.__version__}")
print(f"NumPy:        {np.__version__}")
print(f"Seaborn:       {sns.__version__}")
print('Ambiente configurado com sucesso!')
```

✓ Por que importar tudo de uma vez?

Centralizar as importações no início facilita a leitura e evita erros de "módulo não encontrado" no meio da análise.

random_state=42 é uma convenção: garante que resultados aleatórios sejam reproduzíveis entre execuções.

1

ENTENDIMENTO DO NEGÓCIO

Definir o problema antes de abrir qualquer dado

A primeira fase é conceitual, não envolve código. Seu objetivo é responder com clareza: o que queremos prever ou descobrir? Sem essa definição, qualquer solução estará errada.

1.1 — Perguntas para Definir o Problema

Pergunta	Exemplo Prático
Qual é o objetivo de negócio?	Prever o preço de imóveis em Brasília
O que será previsto (target)?	Coluna preco_mil (valor em R\$ milhares)
Qual o tipo de problema?	REGRESSÃO — prevemos um número contínuo
O que define sucesso?	Erro médio menor que R\$ 100 mil (RMSE < 100)
Quais dados temos disponíveis?	Área, tipo, bairro, quartos, vagas, ano
Há restrições ou limitações?	Outliers de imóveis de luxo acima de R\$ 5M

1.2 — Regressão ou Classificação?

A natureza da variável-alvo (target) define o tipo de algoritmo:

A resposta é...	Tipo de Problema	Exemplos de Algoritmo	Métrica Principal
Um NÚMERO contínuo	REGRESSÃO	LinearRegression, RandomForest	RMSE, MAE, R ²
Uma CATEGORIA / CLASSE	CLASSIFICAÇÃO	LogisticRegression, SVM	Acurácia, F1-Score

⚠ Checklist da Fase 1

- ☐ O problema está claramente definido em uma frase
- ☐ A variável-alvo (target) foi identificada
- ☐ O tipo de problema foi definido (regressão ou classificação)
- ☐ Os critérios de sucesso foram estabelecidos

2

ENTENDIMENTO DOS DADOS

Carregar, explorar e entender o que temos

Nunca modele sem antes explorar os dados. Esta fase — chamada de EDA (Exploratory Data Analysis) — revela problemas, padrões e oportunidades.

2.1 — Carregando os Dados

```
# — CARREGAR DATASET —————
# Substitua o caminho pelo arquivo do seu projeto
df = pd.read_csv('nome_do_seu_arquivo.csv')

# Para rodar no Google Colab (fazer upload manual):
from google.colab import files
uploaded = files.upload()          # abre janela para selecionar arquivo
df = pd.read_csv('nome_do_arquivo.csv')

# Para carregar de URL pública:
# df = pd.read_csv('https://url-do-arquivo.csv')

# — VERIFICAÇÃO RÁPIDA —————
# número de linhas e colunas
print(df.shape)

# nomes das colunas disponíveis
print(df.columns.tolist())

# primeiras 5 linhas — ver como os dados se parecem
df.head()
```

2.2 — Raio-X Completo do Dataset

```
# — df.info() —————
# mostra: tipo de dado por coluna, contagem de não-nulos
# FUNDAMENTAL para detectar colunas com valores ausentes
df.info()

# — df.describe() —————
# estatísticas descritivas de todas as colunas numéricas
# min, max, média, desvio padrão, quartis
# .T transpõe a tabela (facilita a leitura)
df.describe().T.round(2)

# — VALORES ÚNICOS POR COLUNA —————
# quantos valores distintos há em cada coluna
df.nunique()

# contagem de ocorrências de cada categoria
df['coluna_categorica'].value_counts()

# proporção (em vez de contagem)
df['coluna_categorica'].value_counts(normalize=True).round(3)
```

2.3 — Análise de Valores Ausentes (Nulos)

```
# — IDENTIFICAR NULOS —————
# soma de nulos por coluna
nulos = df.isnull().sum()
print(nulos)

# percentual de nulos por coluna
pct_nulos = (df.isnull().sum() / len(df) * 100).round(1)
print(pct_nulos[pct_nulos > 0]) # só mostra colunas COM nulos

# — HEATMAP VISUAL DE NULOS —————
# visualmente mostra onde os nulos estão concentrados
import seaborn as sns
plt.figure(figsize=(10, 4))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis', yticklabels=False)
plt.title('Mapa de Valores Ausentes')
plt.tight_layout()
plt.show()
```

2.4 — Visualização Exploratória Inicial

```
# — DISTRIBUIÇÃO DE UMA VARIÁVEL NUMÉRICA —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# histograma: mostra a distribuição de frequência
axes[0].hist(df['coluna_numerica'], bins=30, color='#2E75B6',
             edgecolor='white', alpha=0.85)
axes[0].axvline(df['coluna_numerica'].mean(), color='red',
               linestyle='--', linewidth=2, label='Média')
axes[0].axvline(df['coluna_numerica'].median(), color='green',
               linestyle=':', linewidth=2, label='Mediana')
axes[0].set_title('Distribuição')
axes[0].set_xlabel('Valor')
axes[0].set_ylabel('Frequência')
axes[0].legend()

# scatter: relação entre duas variáveis numéricas
axes[1].scatter(df['variavel_x'], df['variavel_y'],
               alpha=0.5, color='#1B3A6B', s=40, edgecolor='white')
axes[1].set_title('Variável X vs Variável Y')
axes[1].set_xlabel('Variável X')
axes[1].set_ylabel('Variável Y')

plt.tight_layout()
plt.show()
```

2.5 — Insights Iniciais com groupby

O `groupby` é um dos comandos mais poderosos do Pandas para EDA. Ele agrupa os dados por uma ou mais colunas e aplica uma função de agregação — revelando padrões que as estatísticas globais escondem.

Por que usar groupby na EDA?

A média global pode mascarar diferenças importantes entre grupos.

Exemplo: o preço médio geral é R\$ 800 mil, mas apartamentos custam R\$ 500 mil e coberturas R\$ 2,5 milhões — o `groupby` revela isso.
Use antes de modelar para entender quais categorias mais influenciam o target.

2.5.1 — Agregações Simples por Grupo

```
# — MÉDIA POR CATEGORIA —————
# qual grupo tem o maior valor médio?
df.groupby('coluna_categorica')['coluna_numerica'].mean()
.sort_values(ascending=False)

# — MEDIANA POR CATEGORIA —————
# mais robusta que a média quando há outliers
df.groupby('coluna_categorica')['coluna_numerica'].median()
.sort_values(ascending=False)

# — CONTAGEM POR CATEGORIA —————
# quantos registros há em cada grupo?
df.groupby('coluna_categorica').size().sort_values(ascending=False)

# — VALOR MÁXIMO E MÍNIMO POR GRUPO —————
df.groupby('coluna_categorica')['coluna_numerica'].max()
df.groupby('coluna_categorica')['coluna_numerica'].min()
```

2.5.2 — Múltiplas Funções de Uma Vez com `agg()`

```
# — agg() — VÁRIAS ESTATÍSTICAS SIMULTANEAMENTE —————
# retorna uma tabela com várias métricas por grupo
df.groupby('coluna_categorica')['coluna_numerica'].agg(
    contagem = 'count', # quantos registros
    media    = 'mean',  # média
    mediana  = 'median', # mediana
    minimo   = 'min',   # menor valor
    maximo    = 'max',  # maior valor
    desvio   = 'std',   # desvio padrão
).round(2)

# — FUNÇÕES DIFERENTES POR COLUNA —————
# útil para resumir várias variáveis ao mesmo tempo
df.groupby('coluna_categorica').agg({
    'coluna_numerica1': 'mean',
    'coluna_numerica2': 'median',
    'coluna_numerica3': 'count'
}).round(2)
```

2.5.3 — Agrupamento por Dois Critérios

```
# — AGRUPAR POR DUAS COLUNAS —————
# revela interações entre categorias
df.groupby(['categ1', 'categ2'])['coluna_numerica']
.mean().round(2).sort_values(ascending=False)

# — CONTAR COMBINAÇÕES —————
# quantos registros existem para cada combinação?
df.groupby(['categ1', 'categ2']).size()
```

```
.sort_values(ascending=False)

# — PIVOT TABLE — visão em grade (como Excel) —
# linhas = categ1, colunas = categ2, valores = média
df.pivot_table(values='coluna_numerica',
                index='categ1',
                columns='categ2',
                aggfunc='mean').round(2)
```

2.5.4 — Identificar Grupos com Comportamento Atípico

```
# — DETECTAR GRUPOS COM MAIOR DESVIO DA MÉDIA GERAL —
media_global = df['coluna_numerica'].mean()

medias_grupo = df.groupby('coluna_categorica')['coluna_numerica'].mean()

# diferença percentual de cada grupo em relação à média geral
desvio_pct = ((medias_grupo - media_global) / media_global * 100).round(1)
print(desvio_pct.sort_values(ascending=False))

# — TOP 5 E BOTTOM 5 —
# grupos com os maiores valores
print('--- Top 5 ---')
print(medias_grupo.sort_values(ascending=False).head(5))

# grupos com os menores valores
print('--- Bottom 5 ---')
print(medias_grupo.sort_values(ascending=True).head(5))
```

✓ Boas práticas com groupby na EDA

Sempre combine groupby com sort_values() para destacar os grupos mais relevantes. Use .reset_index() após o groupby para transformar o resultado em DataFrame manipulável. Compare a mediana (robusta) com a média (sensível) — diferenças grandes indicam outliers no grupo. Anote os insights encontrados — eles guiarão as decisões de feature engineering na Fase 3.

⚠ Checklist da Fase 2

- ☐ df.info() executado — tipos de dados verificados
- ☐ df.describe() analisado — outliers suspeitos identificados
- ☐ Nulos contados por coluna (isnull().sum())
- ☐ Distribuição das principais variáveis visualizada
- ☐ groupby executado — padrões por categoria identificados
- ☐ Perguntas iniciais respondidas: quantos registros? quais colunas?

3

PREPARAÇÃO DOS DADOS

Limpar, transformar e criar features

Esta fase consome 60–80% do tempo de um projeto real. Dados sujos produzem modelos ruins — "garbage in, garbage out". Aqui tratamos nulos, outliers, encoding e criamos novas variáveis.

3.1 — Criar uma Cópia de Segurança

```
# SEMPRE trabalhe em uma cópia, nunca modifique o original
# df_raw = dados originais intocados
# df_limpo = cópia para trabalhar
df_limpo = df.copy()

# Para verificar: ambos começam iguais
print(f"Original: {df.shape} | Cópia: {df_limpo.shape}")
```

3.2 — Verificar a Proporção de Valores das Colunas Categóricas

Para colunas do tipo texto (object), é essencial conhecer a distribuição de cada categoria — quais valores existem e em que proporção. O laço abaixo percorre todas as colunas e exibe automaticamente apenas as categóricas.

```
# — PROPORÇÃO DE VALORES POR COLUNA —————
# percorre todas as colunas do DataFrame
for col in df.columns:

    # se a coluna for do tipo texto (object) → categórica
    if df[col].dtype == 'object':
        print('===', col, '===')

        # exibe a proporção de cada categoria em %
        # dropna=False → inclui NaN na contagem
        print(df[col].value_counts(normalize=True, dropna=False).mul(100).round(2))

    # senão → coluna numérica: exibe % preenchido e % faltante
    else:
        print('===', col, '===')

        total = len(df)
        nulos = df[col].isna().sum() # quantidade de nulos
        preenchidos = total - nulos # quantidade preenchida
        porc_preenchidos = round((preenchidos / total) * 100, 2) # % preenchido
        porc_faltantes = round(nulos / total * 100, 2) # % faltante

        print(f'Quantidade de preenchidos: {preenchidos} / Porcentagem de preenchidos: {porc_preenchidos:.2f}%')
        print(f'Quantidade de faltantes: {nulos} / Porcentagem de faltantes: {porc_faltantes:.2f}%')

        # linha separadora entre colunas
        print('=' * 30)
```

i Como interpretar o resultado

`value_counts(normalize=True)` retorna a proporção de cada valor entre 0 e 1.

Multiplicando por 100, obtemos o percentual — ex: 0.45 → 45%.

Use para detectar categorias dominantes, categorias raras ou erros de digitação.

3.2.1 — Corrigindo Valores Inconsistentes em Colunas Categóricas

Ao inspecionar as colunas com `value_counts()`, é comum encontrar variações indesejadas de um mesmo valor — como "Nao", "não", "n", "N". O laço abaixo percorre cada valor da coluna e, se for diferente do valor correto, aplica o `replace()` para substituí-lo.

★ Técnica recomendada — Inspeção + Substituição controlada

Defina o `valor_desejado` como a forma canônica correta.

O laço `for` inspeciona cada linha individualmente.

O `replace()` só é chamado quando o valor é diferente do correto.

Resultado: controle total sobre o que é substituído, sem efeitos colaterais.

```
# — CORRIGIR VALORES INCONSISTENTES —————
# define o valor correto (forma canônica)
valor_desejado = 'nao'

# percorre cada linha da coluna pelo índice e valor
for i, valor in df['col'].items():

    # se o valor for diferente do correto → substituir
    if valor != valor_desejado:
        df['col'] = df['col'].replace(valor, valor_desejado)

# — VERIFICAR O RESULTADO —————
# deve exibir apenas o valor_desejado
print(df['col'].value_counts(dropna=False))
```

✓ Quando usar esta técnica

Após rodar o `value_counts()` no tópico 3.2 e identificar variações indesejadas.

Repita o bloco para cada coluna categórica que apresentar inconsistências.

Execute o `value_counts()` novamente após a correção para confirmar o resultado.

⚠ Atenção — este código NÃO trata valores faltantes (NaN)

No Python, NaN nunca é igual a nada — nem a si mesmo — portanto a condição `if valor != valor_desejado` não o captura de forma confiável.

Valores NaN permanecem intactos após este laço.

Para tratar valores faltantes, utilize `fillna()` conforme o tópico 3.3.

3.2.2 — Identificar e Corrigir Nulos Mascarados

Datasets reais frequentemente representam "ausência de valor" de formas não padronizadas — como ?, -, --, N/D ou espaço em branco. O Pandas não os reconhece como nulos automaticamente, o que faz

com que passem despercebidos no `isnull()`. O tutorial abaixo garante controle total: inspecionar primeiro, decidir, substituir e confirmar.

★ Tutorial — 4 passos para eliminar nulos mascarados

Passo 1 → Inspecionar todas as colunas com `value_counts(dropna=False)`.

Passo 2 → Identificar visualmente os valores suspeitos em cada coluna.

Passo 3 → Substituir os identificados por `np.nan` coluna a coluna.

Passo 4 → Confirmar localmente (por coluna) e globalmente (`isnull().sum()`).

```
# — PASSO 1: INSPECIONAR TODAS AS COLUNAS —————
# rode esta célula e analise a saída com atenção
# procure por valores como: ?, -, --, N/D, none, espaço etc.

for col in df.columns:
    print('===', col, '===')
    print(df[col].value_counts(dropna=False))
    print('=' * 30)

# — PASSO 2: IDENTIFICAR —————
# após analisar a saída do Passo 1, anote quais valores
# são nulos mascarados em cada coluna do seu dataset

# — PASSO 3: SUBSTITUIR POR NaN —————
# importe numpy para usar np.nan (nulo oficial do Pandas)
import numpy as np

# substitua "col" pela coluna real e o valor pelo encontrado no Passo 1
df['col'] = df['col'].replace('?', np.nan) # valor desconhecido
df['col'] = df['col'].replace('-', np.nan) # campo vazio com traço
df['col'] = df['col'].replace('--', np.nan) # variação de traço duplo
df['col'] = df['col'].replace('N/D', np.nan) # não disponível
df['col'] = df['col'].replace(' ', np.nan) # espaço em branco
# repita para cada coluna e valor identificado no Passo 1

# — PASSO 4A: CONFIRMAR POR COLUNA —————
# o NaN deve aparecer na contagem — se não aparecer, revisar o Passo 3
print(df['col'].value_counts(dropna=False))

# — PASSO 4B: CONFIRMAR GLOBALMENTE —————
# agora o isnull() captura todos os nulos, inclusive os mascarados
# use este resultado para guiar o tratamento no tópico 3.3
print(df.isnull().sum())
```

Por que `np.nan` e não `None` ou `string vazia`?

`np.nan` é o padrão reconhecido pelo Pandas em todas as operações: `isnull()`, `fillna()`, `dropna()` e `describe()`.

`None` funciona em colunas object mas pode causar erros em colunas numéricas.

String vazia ("") não é reconhecida como nulo — permaneceria invisível para `isnull()`.

3.3 — Tratamento de Valores Nulos

```

# — REMOVER LINHAS COM NULOS (use com cautela) —————
# recomendado apenas quando a quantidade de nulos é pequena
df_limpo = df_limpo.dropna()

# — PREENCHER COM MEDIANA (recomendado para numéricos) ———
# a mediana é mais robusta que a média em presença de outliers
df_limpo['coluna_numerica'] = df_limpo['coluna_numerica'].fillna(
    df_limpo['coluna_numerica'].median()
)

# — PREENCHER COM MÉDIA —————
df_limpo['coluna_numerica'] = df_limpo['coluna_numerica'].fillna(
    df_limpo['coluna_numerica'].mean()
)

# — PREENCHER COM MODA (para colunas categóricas) —————
# moda = valor mais frequente
df_limpo['coluna_categorica'] = df_limpo['coluna_categorica'].fillna(
    df_limpo['coluna_categorica'].mode()[0]
)

# — CONFIRMAR QUE NÃO HÁ MAIS NULOS —————
print(df_limpo.isnull().sum())

```

3.4 — Detecção e Tratamento de Outliers

```

# — MÉTODO 1: IQR (Intervalo Interquartil) —————
# método estatístico padrão para detectar outliers
Q1 = df_limpo['coluna_numerica'].quantile(0.25)
Q3 = df_limpo['coluna_numerica'].quantile(0.75)
IQR = Q3 - Q1

# limites: valores fora destes são considerados outliers
lim_inf = Q1 - 1.5 * IQR      # limite inferior
lim_sup = Q3 + 1.5 * IQR      # limite superior

n_out = ((df_limpo['coluna_numerica'] < lim_inf) |
         (df_limpo['coluna_numerica'] > lim_sup)).sum()
print(f"Outliers detectados: {n_out}")

# — MÉTODO 2: CORTE POR REGRA DE NEGÓCIO —————
# mais simples e intuitivo quando você conhece o domínio
LIMITE_MAXIMO = 5000          # defina com base no contexto
df_limpo = df_limpo[df_limpo['coluna_numerica'] <= LIMITE_MAXIMO]

# — REMOVER OUTLIERS COM IQR —————
mask = (df_limpo['coluna_numerica'] >= lim_inf) & (df_limpo['coluna_numerica'] <=
lim_sup)
df_sem_outliers = df_limpo[mask].copy()
print(f"Registros antes: {len(df_limpo)} | depois: {len(df_sem_outliers)}")

```

Colinha — Caça Outliers: varredura completa por coluna

Execute este bloco para identificar todos os outliers do dataset de uma vez.
 Percorre cada coluna numérica, calcula o IQR e lista os valores fora dos limites.
 Use antes de decidir o que tratar — veja os valores antes de agir.

```

print('=== CAÇA OUTLIERS - VARREDURA COMPLETA ===')
print('\n')

# percorre todas as colunas do DataFrame
for col in df.columns:

    # aplica apenas em colunas numéricas (ignora 'object')
    if df[col].dtype != 'object':

        # calcular os quartis e o IQR
        Q1 = df[col].quantile(0.25) # 1º quartil (25%)
        Q3 = df[col].quantile(0.75) # 3º quartil (75%)
        IQR = Q3 - Q1                # amplitude interquartil

        # definir os limites
        lim_inf = Q1 - 1.5 * IQR      # abaixo disso = outlier
        lim_sup = Q3 + 1.5 * IQR      # acima disso = outlier

        # máscara booleana - captura TODAS as ocorrências
        # (mais preciso que unique() que ignora frequência)
        mask = (df[col] < lim_inf) | (df[col] > lim_sup)
        n_outliers = mask.sum()      # total de registros
        pct = n_outliers / len(df) * 100 # percentual do dataset

        # exibir resultado
        print(f'Coluna : {col}')
        print(f'Limites: {lim_inf:.2f} → {lim_sup:.2f}')
        print(f'Outliers: {n_outliers} registros ({pct:.2f}%)')

        # listar os valores únicos fora dos limites
        if n_outliers > 0:
            print(f'Valores: {sorted(df[col][mask].unique())}')

print('=' * 40)

```

⚠ Atenção ao corrigir outliers

Identificar não significa remover automaticamente — avalie cada caso.

Um outlier pode ser um erro de digitação (corrigir) ou um dado legítimo e raro (manter).

Sempre trabalhe na cópia `df_limpo`, nunca no `df` original.

3.5 — Encoding de Variáveis Categóricas

```

# — POR QUE FAZER ENCODING? —————
# modelos de ML só aceitam números, não texto
# encoding = converter texto em número

# — ONE-HOT ENCODING (recomendado para maioria dos casos)
# cria uma coluna binária (0 ou 1) para cada categoria
# use quando as categorias NÃO têm ordem (ex: tipo de imóvel)
df_encoded = pd.get_dummies(df_limpo, columns=['coluna_categorica'],
                             drop_first=True) # evitar multicolinearidade

# — LABEL ENCODING (use apenas para árvores de decisão) —
# substitui cada categoria por um número inteiro
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

```

```
df_limpo['coluna_encoded'] = le.fit_transform(df_limpo['coluna_categorica'])

# verificar o mapeamento criado
print(dict(zip(le.classes_, le.transform(le.classes_))))
```

3.6 — Engenharia de Features (Feature Engineering)

```
# — CRIAR NOVAS VARIÁVEIS —————
# novas features podem capturar relações que as originais não capturam

# Exemplo 1: calcular idade a partir do ano de construção
df_limpo['idade'] = 2025 - df_limpo['ano_construcao'].astype(int)

# Exemplo 2: criar razão entre duas variáveis
df_limpo['preco_por_m2'] = df_limpo['preco_mil'] / df_limpo['area_m2']

# Exemplo 3: criar variável binária (flag)
df_limpo['e_novo'] = (df_limpo['idade'] <= 5).astype(int)

# verificar as novas colunas
print(df_limpo[['idade', 'preco_por_m2', 'e_novo']].head())
```

3.7 — Análise de Correlação de Pearson



O que é a Correlação de Pearson?

É uma medida estatística que indica a força e a direção da relação linear entre duas variáveis numéricas. O resultado é o coeficiente r , que varia de -1 a +1.

$r = +1$ → relação positiva perfeita: quando X sobe, Y sobe na mesma proporção.

$r = -1$ → relação negativa perfeita: quando X sobe, Y desce na mesma proporção.

$r = 0$ → sem relação linear entre as variáveis.

Em Machine Learning, usamos r para selecionar quais variáveis têm mais relação com o target — quanto maior o $|r|$, mais útil a feature tende a ser para o modelo.



O que é um Heatmap de Correlação?

É uma representação visual da matriz de correlação — uma tabela onde cada célula mostra o valor r entre dois pares de variáveis, colorida por intensidade.

Cores quentes (vermelho) → correlação positiva forte (próximo de +1).

Cores frias (azul) → correlação negativa forte (próximo de -1).

Cores neutras (branco/bege) → sem correlação linear (próximo de 0).

A diagonal principal sempre vale 1,00 — toda variável tem correlação perfeita consigo mesma.

```
# — HEATMAP DE CORRELAÇÃO —————
# visualização da matriz completa entre todas as variáveis
numericas = df_limpo.select_dtypes(include='number')
plt.figure(figsize=(10, 7))
sns.heatmap(numericas.corr(), annot=True, fmt='.2f',
            cmap='coolwarm', center=0,
            linewidths=0.5, annot_kws={'size': 9})
plt.title('Matriz de Correlação de Pearson', fontsize=13)
plt.tight_layout()
plt.show()
```

3.7.1 — Tabela de Seleção de Features por Correlação

O script abaixo percorre todas as features numéricas, calcula a correlação de cada uma com o target e retorna uma tabela ordenada da mais à menos correlacionada — com classificação e recomendação de uso no modelo.

```
# — PASSO 1: DEFINIR O TARGET —————
# substitua pelo nome real da sua coluna-alvo
TARGET = 'coluna_alvo'

# — PASSO 2: SELECIONAR COLUNAS NUMÉRICAS —————
# a correlação de Pearson só funciona com variáveis numéricas
numericas = df_limpo.select_dtypes(include='number')

# — PASSO 3: FUNÇÃO DE CLASSIFICAÇÃO —————
# recebe o valor absoluto de r e retorna classificação textual
def classificar(r):
    if r >= 0.90: return 'Muito forte'
    elif r >= 0.70: return 'Forte'
    elif r >= 0.50: return 'Moderada'
    elif r >= 0.30: return 'Fraca'
    else:         return 'Muito fraca'

# — PASSO 4: CALCULAR CORRELAÇÃO COM O TARGET —————
# .corr() calcula a matriz completa de correlações
# [TARGET] seleciona apenas a coluna do target
# .drop(TARGET) remove a correlação do target consigo mesmo (sempre 1.0)
correlacao = numericas.corr()[TARGET].drop(TARGET)

# — PASSO 5: MONTAR TABELA DE RESULTADOS —————
tabela = pd.DataFrame({
    'feature':      correlacao.index,
    'r':            correlacao.values.round(4),
    'r_absoluto':   correlacao.abs().values.round(4),
    'classificacao': correlacao.abs().apply(classificar),
    'recomendacao': correlacao.abs().apply(
        lambda r: 'Incluir' if r >= 0.30 else 'Descartar'
    )
})

# — PASSO 6: ORDENAR DO MAIS AO MENOS CORRELACIONADO —————
tabela = tabela.sort_values('r_absoluto', ascending=False).reset_index(drop=True)

# — PASSO 7: EXIBIR A TABELA —————
print(tabela.to_string(index=False))
```

Como usar a tabela para selecionar features

Inclua no modelo as features com recomendacao = Incluir ($|r| \geq 0,30$).

Correlações negativas são tão válidas quanto positivas — o que importa é o `r_absoluto`.

Features com `r_absoluto` muito próximas entre si podem ser redundantes — avalie remover uma.

Após a seleção, volte ao tópico 5.1 e defina FEATURES com as colunas escolhidas.

Valor de r	Classificação	Recomendação
0,90 a 1,00	Muito forte	Incluir — excelente feature
0,70 a 0,89	Forte	Incluir — boa feature

0,50 a 0,69	Moderada	Incluir — feature razoável
0,30 a 0,49	Fraca	Incluir com cautela
0,00 a 0,29	Muito fraca	Descartar
Valores negativos	Mesma lógica, relação inversa	$r = -0,85$ é tão forte quanto $r = +0,85$

⚠ Checklist da Fase 3

- ☐ Cópia criada (`df_limpo = df.copy()`)
- ☐ Nulos mascarados identificados e convertidos para NaN (3.2.2)
- ☐ Valores inconsistentes corrigidos (3.2.1)
- ☐ Nulos tratados com `fillna()` ou `dropna()` (3.3)
- ☐ Outliers identificados e tratados (3.4)
- ☐ Variáveis categóricas convertidas para números (3.5)
- ☐ Novas features criadas se relevante (3.6)
- ☐ Tabela de correlação gerada e features selecionadas (3.7)

4

VISUALIZAÇÃO DE DADOS

Gráficos sns e plt para análise exploratória

Antes de modelar, visualizar os dados é essencial para entender distribuições, relações e padrões. Os gráficos abaixo são os mais utilizados nas aulas e exercícios — todos aplicáveis diretamente ao seu dataset.

Tabela de Referência — Tipos de Gráfico

Gráfico	Biblioteca	Quando usar	Finalidade	Sintaxe básica
hist / histplot	plt / sns	Distribuição de uma variável numérica	Ver concentração, assimetria e outliers	plt.hist(df['col'], bins=30)
scatter / scatterplot	plt / sns	Relação entre duas variáveis numéricas	Verificar correlação visual e presença de outliers	plt.scatter(df['x'], df['y'])
boxplot	sns	Comparar distribuição entre grupos	Ver mediana, quartis e outliers por categoria	sns.boxplot(data=df, x='cat', y='num')
violinplot	sns	Distribuição completa por grupo	Combina boxplot com curva de densidade	sns.violinplot(data=df, x='cat', y='num')
barplot	sns	Comparar médias entre categorias	Média com intervalo de confiança por grupo	sns.barplot(data=df, x='cat', y='num')
heatmap	sns	Correlação entre variáveis numéricas	Identificar quais features se relacionam com o target	sns.heatmap(df.corr(), annot=True)
lmplot	sns	Regressão visual por grupo	Ver linha de tendência e dispersão por categoria	sns.lmplot(data=df, x='x', y='y', hue='cat')

Histograma — plt.hist / sns.histplot

Mostra como os valores de uma variável numérica se distribuem. A linha vermelha indica a média e a verde a mediana — quando diferem muito, há assimetria ou outliers.

```
# — HISTOGRAMA COM MATPLOTLIB —————
fig, ax = plt.subplots(figsize=(10, 5))

ax.hist(df['coluna_numerica'], bins=30,
        color='#2E75B6', edgecolor='white', alpha=0.85)

# linha da média
ax.axvline(df['coluna_numerica'].mean(),
           color='red', linestyle='--', linewidth=2, label='Média')
# linha da mediana
ax.axvline(df['coluna_numerica'].median(),
           color='green', linestyle=':', linewidth=2, label='Mediana')

ax.set_title('Distribuição de coluna_numerica')
ax.set_xlabel('Valor')
ax.set_ylabel('Frequência')
```

```
ax.legend()
plt.tight_layout()
plt.show()

# — HISTPLOT COM SEABORN (com KDE e hue) —————
# kde=True → curva de densidade suavizada
# hue → separa por categoria com cores diferentes
sns.histplot(data=df, x='coluna_numerica',
              hue='coluna_categorica',
              kde=True, element='step')
plt.title('Distribuição por Categoria')
plt.show()
```

Scatter — plt.scatter / sns.scatterplot

Plota um ponto por registro relacionando duas variáveis numéricas. Ideal para verificar correlação visual — pontos em diagonal indicam relação, pontos dispersos indicam ausência de relação.

```
# — SCATTER COM MATPLOTLIB —————
fig, ax = plt.subplots(figsize=(10, 6))

ax.scatter(df['variavel_x'], df['variavel_y'],
           alpha=0.5, color='#1B3A6B',
           s=40, edgecolors='white')

ax.set_title('Variável X vs Variável Y')
ax.set_xlabel('Variável X')
ax.set_ylabel('Variável Y')
plt.tight_layout()
plt.show()

# — SCATTERPLOT COM SEABORN (com hue por categoria) —————
# hue → colore os pontos por categoria
sns.scatterplot(data=df, x='variavel_x', y='variavel_y',
                hue='coluna_categorica',
                alpha=0.6, s=60)
plt.title('X vs Y por Categoria')
plt.show()
```

Boxplot — sns.boxplot

Resume a distribuição de uma variável numérica por grupo. A caixa vai do Q1 ao Q3 (50% dos dados), a linha central é a mediana e os pontos fora dos bigodes são outliers.

```
# — BOXPLOT COM SEABORN —————
fig, ax = plt.subplots(figsize=(10, 6))

sns.boxplot(data=df, x='coluna_categorica', y='coluna_numerica',
            order=['Cat_A', 'Cat_B', 'Cat_C'], # ordem das categorias
            palette='Set2', ax=ax)

ax.set_title('Distribuição por Grupo')
ax.set_xlabel('Categoria')
ax.set_ylabel('Valor')
plt.tight_layout()
plt.show()
```

Violinplot — sns.violinplot

Versão aprimorada do boxplot — além de mostrar Q1, mediana e Q3, exibe a curva de densidade completa. A largura do "violino" indica onde os dados são mais concentrados.

```
# — VIOLINPLOT COM SEABORN —————
fig, ax = plt.subplots(figsize=(10, 6))

sns.violinplot(data=df, x='coluna_categorica', y='coluna_numerica',
               inner='box',          # inner: box, quartile, point, stick
               palette='Set2', ax=ax)

ax.set_title('Distribuição Completa por Grupo')
ax.set_xlabel('Categoria')
ax.set_ylabel('Valor')
plt.tight_layout()
plt.show()
```

Barplot — sns.barplot

Compara a média de uma variável numérica entre categorias. As barras de erro mostram o intervalo de confiança de 95% — útil para avaliar se as diferenças entre grupos são estatisticamente relevantes.

```
# — BARPLOT COM SEABORN —————
fig, ax = plt.subplots(figsize=(10, 6))

sns.barplot(data=df, x='coluna_categorica', y='coluna_numerica',
            estimator='mean',      # função de agregação: mean, median, sum
            errorbar='ci',         # barra de erro: intervalo de confiança 95%
            palette='Blues_d', ax=ax)

ax.set_title('Média por Categoria')
ax.set_xlabel('Categoria')
ax.set_ylabel('Média')
plt.xticks(rotation=45) # rotaciona rótulos se forem longos
plt.tight_layout()
plt.show()
```

Heatmap de Correlação — sns.heatmap

Visualiza a matriz de correlação de Pearson. Cada célula mostra o valor r entre dois pares de variáveis, colorida por intensidade. Indispensável antes de selecionar features para o modelo.

```
# — HEATMAP DE CORRELAÇÃO —————
# selecionar apenas colunas numéricas
numericas = df.select_dtypes(include='number')

plt.figure(figsize=(10, 7))
sns.heatmap(numericas.corr(),
            annot=True,          # exibe o valor r em cada célula
            fmt='.2f',           # formato: 2 casas decimais
            cmap='coolwarm',     # paleta: azul (negativo) → vermelho (positivo)
            center=0,            # centraliza a escala no zero
            linewidths=0.5,      # linhas separadoras entre células
            annot_kws={'size': 9})

plt.title('Matriz de Correlação de Pearson', fontsize=13)
plt.tight_layout()
```

```
plt.show()
```

Lmplot — sns.lmplot

Combina scatter com linha de regressão linear. O parâmetro `hue` permite traçar uma reta por categoria — revelando se a relação entre X e Y muda conforme o grupo.

```
# — LMPLLOT COM SEABORN —————
# lmplot opera diretamente no DataFrame — não usa fig/ax
g = sns.lmplot(data=df, x='variavel_x', y='variavel_y',
               hue='coluna_categorica', # reta por categoria
               height=5, aspect=2,
               scatter_kws={'alpha': 0.35, 's': 35},
               line_kws={'linewidth': 2.5},
               ci=95) # intervalo de confiança da reta

g.figure.suptitle('Regressão Visual: X vs Y por Categoria', y=1.02)
g.set_axis_labels('Variável X', 'Variável Y')
plt.tight_layout()
plt.show()
```

Subplots — Múltiplos Gráficos em uma Figura

Use `plt.subplots()` para combinar vários gráficos em uma única figura — muito útil para comparar distribuições lado a lado sem abrir várias janelas.

```
# — MÚLTIPLOS GRÁFICOS LADO A LADO —————
# cria 1 linha e 2 colunas de gráficos
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Análise Exploratória', fontsize=13, fontweight='bold')

# gráfico da esquerda: histograma
axes[0].hist(df['coluna_numerica'], bins=30,
             color='#2E75B6', edgecolor='white')
axes[0].set_title('Distribuição')
axes[0].set_xlabel('Valor')
axes[0].set_ylabel('Frequência')

# gráfico da direita: scatter
axes[1].scatter(df['variavel_x'], df['variavel_y'],
               alpha=0.5, color='#C2185B', s=40)
axes[1].set_title('X vs Y')
axes[1].set_xlabel('Variável X')
axes[1].set_ylabel('Variável Y')

plt.tight_layout()
plt.show()
```

✓ Boas práticas de visualização

Sempre defina título (`set_title`), rótulos dos eixos (`set_xlabel`, `set_ylabel`) e legenda (`legend()`) — gráficos sem identificação não comunicam.

Use `alpha` entre 0.4 e 0.7 em scatter e histogramas para visualizar sobreposição de pontos.

`plt.tight_layout()` evita que os gráficos se sobreponham quando usar subplots.

`sns.set_theme(style="whitegrid")` melhora a legibilidade — execute uma vez no início do notebook.

5

MODELAGEM

Dividir dados, treinar e ajustar o modelo

5.1 — Separar Features (X) e Target (y)

```
# — DEFINIR FEATURES E TARGET —————
# X = variáveis de entrada (o que o modelo usa para prever)
# y = variável-alvo (o que queremos prever)

# features numéricas que serão usadas no modelo
FEATURES = ['coluna1', 'coluna2', 'coluna3']
TARGET = 'coluna_alvo'

# criar X e y
X = df_limpo[FEATURES]
y = df_limpo[TARGET]

print(f"X shape: {X.shape} → {X.shape[0]} exemplos, {X.shape[1]} features")
print(f"y shape: {y.shape} → {y.shape[0]} valores-alvo")
```

5.2 — Divisão Treino e Teste

Dividir os dados é obrigatório. O modelo aprende nos dados de treino e é avaliado nos dados de teste — que ele nunca viu. Avaliar no treino seria como "colar na prova".

```
# — train_test_split —————
from sklearn.model_selection import train_test_split

X_treino, X_teste, y_treino, y_teste = train_test_split(
    X,          # features
    y,          # target
    test_size=0.2, # 20% para teste, 80% para treino
    random_state=42 # semente: garante reprodutibilidade
)

# verificar os tamanhos
print(f'Treino: {X_treino.shape[0]} exemplos ({X_treino.shape[0]/len(X)*100:.0f}%)')
print(f'Teste: {X_teste.shape[0]} exemplos ({X_teste.shape[0]/len(X)*100:.0f}%)')
```

i Por que random_state=42?

A divisão treino/teste é aleatória. random_state garante que, ao rodar de novo, os mesmos dados vão para treino e os mesmos para teste.

Você pode usar qualquer número inteiro — 42 é uma convenção popular na comunidade de dados.

⚠ Checklist da Fase 5

- ☐ Features e target separados (X e y)
- ☐ Dados divididos em treino e teste (80/20)
- ☐ random_state definido para reprodutibilidade

6

REGRESSÃO LINEAR

Simples, múltipla, seleção de features e avaliação

6.1 — Regressão Linear: Simples e Múltipla



Regressão Linear Simples

Usa apenas 1 feature para prever o target. A relação é representada por uma reta no plano.

Equação: $y = a + b \cdot x$

a = intercepto (valor de y quando x = 0)

b = coeficiente (quanto y muda a cada 1 unidade de x)

Quando usar:

- ✓ Relação clara entre 1 feature e o target
- ✓ Análise inicial ou exploratória

X Limitação: ignora todas as outras variáveis

Exemplo: prever preço de imóvel usando apenas área_m2.



Regressão Linear Múltipla

Usa 2 ou mais features simultaneamente. Cada variável contribui com seu peso para a previsão.

Equação: $y = a + b_1x_1 + b_2x_2 + \dots + b_nx_n$

a = intercepto

$b_1 \dots b_n$ = coeficiente de cada feature

Quando usar:

- ✓ Múltiplas variáveis influenciam o target
- ✓ Maior precisão que a simples na maioria dos casos
- ✓ Features selecionadas pela correlação (tópico 3.7)

Exemplo: prever preço usando área_m2, quartos, vagas e distância.

6.1.1 — Script: Regressão Linear Simples

```
# — REGRESSÃO LINEAR SIMPLES — 1 FEATURE —
from sklearn.linear_model import LinearRegression

# definir 1 única feature e o target
FEATURES = ['coluna_numerica'] # apenas 1 variável de entrada
TARGET = 'coluna_alvo'

# separar X e y
X = df_limpo[FEATURES]
y = df_limpo[TARGET]

# dividir treino e teste
from sklearn.model_selection import train_test_split
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.2, random_state=42)

# criar e treinar o modelo
modelo_simples = LinearRegression()
modelo_simples.fit(X_treino, y_treino)

# gerar previsões
y_pred_treino = modelo_simples.predict(X_treino)
y_pred_teste = modelo_simples.predict(X_teste)

# — IMPRIMIR EQUAÇÃO —
# modelo.coef_[0] = coeficiente (inclinação da reta)
# modelo.intercept_ = intercepto (valor de y quando x = 0)
```



```
print(f'Equação do modelo: y = {modelo_simples.intercept_:.2f} +  
{modelo_simples.coef_[0]:.4f}x')
```

Atributo	O que representa	Na equação	Exemplo prático
modelo.coef_[0]	Coefficiente — o quanto o target muda a cada 1 unidade da feature. Pode ser positivo (relação direta) ou negativo (relação inversa).	b em $y = a + bx$	coef_ = 0.87 → a cada 1 m ² a mais, o preço sobe R\$ 0,87 mil
modelo.intercept_	Intercepto — valor previsto pelo modelo quando a feature vale zero. Nem sempre tem interpretação prática, mas é necessário para a equação.	a em $y = a + bx$	intercept_ = 1250.50 → ponto de partida da reta no eixo y

Por que coef_ tem colchete [0]?

O sklearn armazena os coeficientes em uma lista — um por feature.

Na regressão simples há apenas 1 feature, então o coeficiente está na posição [0].

Na regressão múltipla, modelo.coef_ retorna uma lista com um valor para cada feature — por isso usamos zip(FEATURES, modelo.coef_) para imprimir todos.

6.1.2 — Script: Regressão Linear Múltipla

```
# — DEFINIR FEATURES E TARGET —————
# use as features selecionadas na tabela de correlação (tópico 3.7)
FEATURES = ['coluna1', 'coluna2', 'coluna3', 'coluna4']
TARGET = 'coluna_alvo'

# separar X e y
X = df_limpo[FEATURES]
y = df_limpo[TARGET]

# dividir treino e teste
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.2, random_state=42)

# — TREINAR —————
modelo_mult = LinearRegression()
modelo_mult.fit(X_treino, y_treino)

# — PREVER —————
y_pred_treino = modelo_mult.predict(X_treino)
y_pred_teste = modelo_mult.predict(X_teste)

# — GERAR EQUAÇÃO —————
print('Equação do modelo:\n')
for feat, coef in zip(FEATURES, modelo_mult.coef_):
    print(f'{feat}: {coef:.2f}')
    print('-' * 20)
print(f'Intercepto: {modelo_mult.intercept_:.2f}')
```

Por que a múltipla geralmente supera a simples?

O preço de um imóvel não depende só da área — quartos, vagas, localização e idade do imóvel também importam.

A regressão simples ignora essas variáveis, enquanto a múltipla pondera cada uma com seu coeficiente.

Regra prática: comece com a simples para entender a relação principal, depois expanda para a múltipla com as features do tópico 3.7.



Preparando para a próxima técnica: Classificação

A regressão linear prevê um número contínuo. A classificação prevê uma categoria. A lógica de treino, divisão treino/teste, features e avaliação é a mesma — o que muda é o algoritmo e as métricas.

Regressão → target é um **número contínuo** → Ex: prever o **preço** de um imóvel

Classificação → target é uma **categoria** → Ex: prever se o imóvel **será vendido ou não**

Toda a preparação dos dados (Fases 3, 3.7) e a divisão treino/teste (5.1, 5.2) são reutilizadas na classificação — o investimento nessas etapas vale para qualquer modelo supervisionado.

6.2 — Seleção Final de Features e Comparação de Modelos

Incluir todas as features disponíveis não é necessariamente a melhor estratégia. Esta seção explica por que features desnecessárias podem prejudicar o modelo, o que significam R^2 e RMSE, e como comparar modelos para tomar a melhor decisão.

⚠ Por que features desnecessárias prejudicam o modelo?

1. Ruído — features sem relação com o target ensinam padrões inexistentes ao modelo.
2. Multicolinearidade — duas features muito correlacionadas entre si ($r > 0.85$) distorcem os coeficientes: o modelo não consegue separar o efeito de cada uma.
3. Overfitting — mais features = mais parâmetros = maior risco de decorar o treino em vez de aprender o padrão real.

Regra Prática de Decisão

Situação	Decisão
r_absoluto com target ≥ 0.30	Incluir no modelo
r_absoluto com target < 0.30	Descartar — teste sem ela
Duas features com $r > 0.85$ entre si	Manter apenas 1 das duas
Feature relevante mas com muitos nulos	Avaliar se vale tratar ou descartar
Modelo mais simples com R^2 similar	Preferir o mais simples — princípio da parcimônia

Script de Comparação entre Modelos

Treine três versões do modelo e compare R^2 e RMSE no teste para decidir quais features realmente contribuem.

```
# — IMPORTAÇÕES —
from sklearn.linear_model import LinearRegression
```

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
import pandas as pd

# — DEFINIR OS GRUPOS DE FEATURES —
# Modelo A — todas as features disponíveis
FEATURES_A = ['col1', 'col2', 'col3', 'col4', 'col5']

# Modelo B — apenas features com r_absoluto >= 0.30 (tópico 3.7)
FEATURES_B = ['col1', 'col2', 'col3']

# Modelo C — apenas as 2 mais correlacionadas
FEATURES_C = ['col1', 'col2']

TARGET = 'coluna_alvo'

# — FUNÇÃO AUXILIAR DE TREINO E AVALIAÇÃO —
# evita repetir o mesmo bloco 3 vezes
def treinar_avaliar(features, nome):
    X = df_limpo[features]
    y = df_limpo[TARGET]

    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.2, random_state=42)

    modelo = LinearRegression()
    modelo.fit(X_tr, y_tr)

    y_pred_tr = modelo.predict(X_tr)
    y_pred_te = modelo.predict(X_te)

    return {
        'Modelo': nome,
        'Nº Features': len(features),
        'R² Treino': round(r2_score(y_tr, y_pred_tr), 4),
        'R² Teste': round(r2_score(y_te, y_pred_te), 4),
        'RMSE Treino': round(np.sqrt(mean_squared_error(y_tr, y_pred_tr)), 2),
        'RMSE Teste': round(np.sqrt(mean_squared_error(y_te, y_pred_te)), 2),
    }

# — TREINAR OS TRÊS MODELOS —
resultado_A = treinar_avaliar(FEATURES_A, 'Modelo A — todas')
resultado_B = treinar_avaliar(FEATURES_B, 'Modelo B — r >= 0.30')
resultado_C = treinar_avaliar(FEATURES_C, 'Modelo C — top 2')

# — TABELA COMPARATIVA —
comparacao = pd.DataFrame([resultado_A, resultado_B, resultado_C])
print(comparacao.to_string(index=False))

```

✓ Como decidir qual modelo usar?

Menor RMSE no teste → previsão mais precisa na prática.

Maior R² no teste → melhor explicação da variância.

Se dois modelos têm R² e RMSE similares → prefira o mais simples (menos features).

Se R² treino >> R² teste → overfitting: remova features ou reduza a complexidade.

Princípio da parcimônia: o melhor modelo é o mais simples que resolve bem o problema.

6.3 — Avaliação do Modelo

Após treinar, é hora de medir. Sempre calcule as métricas nos dados de TREINO e de TESTE e compare os dois — essa diferença é a chave para detectar overfitting.

Antes de calcular, é fundamental entender o que cada métrica mede e como interpretá-la. Sempre calcule nos dados de TREINO e de TESTE e compare os dois — essa diferença é a chave para detectar overfitting.

R² — o modelo é bom?

Responde: "quanto da variação do target o modelo consegue explicar?" Vai de 0 a 1 — quanto mais perto de 1, melhor.

`r2_score(y_real, y_pred)`

R² = 1.00 → explica 100% da variação — perfeito (desconfie!)

R² = 0.85 → explica 85% — muito bom

R² = 0.50 → explica 50% — razoável

R² = 0.00 → não explica nada — inútil

Atenção: R² treino muito maior que R² teste → modelo decorou o treino (overfitting).

RMSE — quanto erro estou cometendo?

Responde: "em média, quanto o modelo erra?" O resultado vem na mesma unidade do target — fácil de interpretar no contexto do problema.

`np.sqrt(mean_squared_error(y_real, y_pred))`

RMSE = 50.000 → o modelo erra em média R\$ 50 mil
Penaliza erros grandes — um erro de 100k pesa mais que dois erros de 50k. Quanto menor, melhor.

Atenção: RMSE teste > RMSE treino em mais de 20% → modelo não generaliza (overfitting).

MAE — Erro Médio Absoluto (complementar)

MAE = média dos erros absolutos, sem elevar ao quadrado — trata todos os erros da mesma forma. Menos sensível a outliers que o RMSE: se houver previsões muito erradas, o RMSE sobe mais que o MAE.

Use MAE quando o dataset tem outliers que poderiam distorcer a análise pelo RMSE.

Regra prática: calcule os três (R², RMSE e MAE) e compare — se RMSE >> MAE, há outliers nos erros.

Métrica	O que mede	Unidade	Sensível a outliers?	Quando priorizar
R ²	Proporção da variância explicada	Adimensional (0 a 1)	Moderado	Comparar modelos entre si
RMSE	Erro médio — penaliza erros grandes	Mesma do target	Sim	Quando erros grandes são inaceitáveis
MAE	Erro médio absoluto — todos erros iguais	Mesma do target	Não	Quando há outliers nos dados

6.3.1 — Calcular e Comparar Métricas

```
# — IMPORTAR FUNÇÕES DE MÉTRICAS —————
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import numpy as np

# — CALCULAR NO TREINO —————
rmse_treino = np.sqrt(mean_squared_error(y_treino, y_pred_treino))
mae_treino  = mean_absolute_error(y_treino, y_pred_treino)
r2_treino   = r2_score(y_treino, y_pred_treino)

# — CALCULAR NO TESTE —————
```

```

rmse_teste = np.sqrt(mean_squared_error(y_teste, y_pred_teste))
mae_teste  = mean_absolute_error(y_teste, y_pred_teste)
r2_teste   = r2_score(y_teste, y_pred_teste)

# — TABELA COMPARATIVA —
print("=" * 50)
print("COMPARAÇÃO: TREINO vs TESTE")
print("=" * 50)
print(f"  RMSE   Treino: {rmse_treino:.2f} | Teste: {rmse_teste:.2f}")
print(f"  MAE    Treino: {mae_treino:.2f} | Teste: {mae_teste:.2f}")
print(f"  R²     Treino: {r2_treino:.4f} | Teste: {r2_teste:.4f}")

# — DIAGNÓSTICO —
dif_rmse = (rmse_teste - rmse_treino) / rmse_treino * 100
if dif_rmse < 10:
    print("DIAGNÓSTICO: Bom ajuste – modelo generaliza bem")
elif dif_rmse < 20:
    print("DIAGNÓSTICO: Leve overfitting – aceitável")
else:
    print("DIAGNÓSTICO: OVERFITTING – modelo decorou o treino")

```

6.3.2 — Gráfico Real vs Previsto e Resíduos

```

# — GRÁFICO DIAGNÓSTICO COMPLETO —
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Gráfico 1: Real vs Previsto
# pontos na diagonal = previsões perfeitas
axes[0].scatter(y_teste, y_pred_teste, alpha=0.6,
                color='#0066CC', edgecolor='white', s=50)

# linha de referência (previsão perfeita)
ref = [y_teste.min(), y_teste.max()]
axes[0].plot(ref, ref, 'r--', linewidth=2, label='Previsão Perfeita')
axes[0].set_xlabel('Valor Real')
axes[0].set_ylabel('Valor Previsto')
axes[0].set_title('Real vs Previsto – Dados de TESTE')
axes[0].legend()

# Gráfico 2: Resíduos (erros)
# resíduos = diferença entre real e previsto
# um bom modelo tem resíduos aleatórios, próximos de zero
residuos = y_teste - y_pred_teste
axes[1].scatter(y_pred_teste, residuos, alpha=0.5,
                color='#C2185B', edgecolor='white', s=50)
axes[1].axhline(0, color='black', linestyle='--', linewidth=1.5)
axes[1].set_xlabel('Valor Previsto')
axes[1].set_ylabel('Resíduo (Real - Previsto)')
axes[1].set_title('Análise de Resíduos')

plt.tight_layout()
plt.show()

```

6.3.3 — Diagnóstico de Overfitting

Situação	RMSE Treino	RMSE Teste	Diagnóstico	Ação
Ideal	Baixo	Baixo (similar)	Modelo OK	Seguir para deploy

Overfitting	Muito baixo	Alto (>20% maior)	Modelo decorou	Simplificar modelo, regularizar
Underfitting	Alto	Alto	Modelo fraco	Adicionar features, modelo mais complexo

⚠ Checklist da Fase 6 — Regressão Linear

- ☐ Tipo de regressão escolhido: simples (1 feature) ou múltipla (n features)
- ☐ Modelos A, B e C treinados e comparados (6.2)
- ☐ Melhor modelo selecionado com base no R^2 e RMSE no teste
- ☐ Multicolinearidade verificada entre as features selecionadas
- ☐ Previsões geradas para treino E teste
- ☐ RMSE, MAE e R^2 calculados no treino e no teste (6.3.1)
- ☐ Gráfico Real vs Previsto gerado (6.3.2)
- ☐ Gráfico de resíduos gerado (6.3.2)
- ☐ Diagnóstico de overfitting feito (6.3.3)

7

CLASSIFICAÇÃO

KNN · Árvore de Decisão · Naive Bayes · Regressão Logística

7.1 — O que é Classificação?

Classificação é a tarefa de treinar um modelo para aprender a relação entre features de entrada e categorias predefinidas — e, a partir disso, prever a qual classe um novo dado pertence. Diferente da Regressão, que produz um número contínuo, a Classificação produz uma saída discreta: o modelo escolhe entre categorias.

i Saída de um Classificador

Na maioria dos algoritmos, o modelo NÃO retorna apenas a classe. Ele retorna uma probabilidade para cada categoria, e a classe com maior probabilidade é escolhida. Exemplo: [Maligno: 18%, Benigno: 82%] → Classificado como Benigno.

A resposta é...	Tipo de Problema	Exemplos de Algoritmo	Métrica Principal
Um NÚMERO contínuo	REGRESSÃO	LinearRegression, RandomForest	RMSE, MAE, R^2
Uma CATEGORIA / CLASSE	CLASSIFICAÇÃO	KNN, DecisionTree, LogisticRegression	Acurácia, F1-Score

Tipos de Problemas de Classificação

Tipo	Descrição e Exemplo
Binária	Duas classes: spam / não-spam, fraude / legítimo, tumor maligno / benigno. Saída: 0 ou 1.
Multiclasse	Três ou mais classes exclusivas. Ex: diagnóstico → gripe, COVID, alergia, resfriado.
Multilabel	Uma amostra pode pertencer a várias classes. Ex: notícia classificada como política + economia + internacional.
Desbalanceada	Uma classe tem muito mais exemplos que outras. Ex: fraude = 0,1% das transações. Cuidado: acurácia pode ser enganosa!

Aplicações no Mundo Real

Domínio	Algoritmo e Classificação
Filtro de Spam	Naive Bayes → spam / não-spam
Fraude Financeira	Random Forest → fraude / legítimo
Diagnóstico Médico	Árvore de Decisão → classificação de doenças ou riscos
Análise de Sentimentos	Naive Bayes / BERT → positivo, negativo, neutro
Marketing e Churn	Regressão Logística → vai cancelar / não vai cancelar
Visão Computacional	Redes Neurais → classificação de imagens e objetos

Aprovação de Crédito

Árvore de Decisão / KNN → aprovado / negado

7.2 — Tipos de Dados e Encoding

Algoritmos de Machine Learning trabalham apenas com números. Dados categóricos precisam ser convertidos antes do treino.

Tipo	Exemplos e Observações
Quantitativo Discreto	Número de filhos, cliques, cursos. Valores inteiros contáveis.
Quantitativo Contínuo	Altura, peso, salário, temperatura. Qualquer valor em um intervalo.
Qualitativo Nominal	Sexo, cor dos olhos, tipo sanguíneo, país. SEM ordem.
Qualitativo Ordinal	Escolaridade, classe social, grau de satisfação. COM hierarquia.

Label Encoding — Ordinais

Atribui um número inteiro a cada categoria, preservando a ordem natural. Use APENAS para variáveis com hierarquia real.

```
# — LABEL ENCODING PARA VARIÁVEL ORDINAL —————
# define o mapeamento manual preservando a hierarquia
mapa_escolaridade = {
    'Fundamental': 0, 'Médio': 1, 'Superior': 2, 'Pós-graduação': 3
}
df['escolaridade_num'] = df['escolaridade'].map(mapa_escolaridade)

# — LABEL ENCODING BINÁRIO (sim/não, masculino/feminino) —
df['sexo_num'] = (df['sexo'] == 'Masculino').astype(int)
df['restricao_num'] = (df['restricao_spc'] == 'Sim').astype(int)
```

One-Hot Encoding — Nominais

Cria uma coluna binária para cada categoria. Use para variáveis nominais (sem ordem). Descarte uma coluna para evitar multicolinearidade.

```
# — ONE-HOT ENCODING COM PANDAS —————
# drop_first=True descarta uma coluna para evitar multicolinearidade
df_encoded = pd.get_dummies(df, columns=['estado_civil', 'regiao'],
                             drop_first=True)

# — ALTERNATIVA COM SKLEARN (preferível em pipelines) —
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(drop='first', sparse_output=False)
encoded = enc.fit_transform(df[['estado_civil']])
```

⚠ Cuidado com Label Encoding em Nominais

Aplicar Label Encoding em variáveis sem ordem (ex: norte=0, sul=1, leste=2) cria uma falsa hierarquia que confunde o modelo.

Para variáveis nominais com mais de 2 valores, prefira One-Hot Encoding.

7.3 — Métricas de Avaliação

A avaliação de um classificador vai além da acurácia. Em classes desbalanceadas, um modelo pode atingir 99% de acurácia simplesmente prevendo sempre a classe majoritária.

Matriz de Confusão

Resultado	Descrição
Verdadeiro Positivo (VP)	Modelo previu POSITIVO e estava correto. Ex: tumor maligno identificado.
Verdadeiro Negativo (VN)	Modelo previu NEGATIVO e estava correto. Ex: tumor benigno identificado.
Falso Positivo (FP)	Modelo previu POSITIVO mas estava errado. Ex: benigno classificado como maligno.
Falso Negativo (FN)	Modelo previu NEGATIVO mas estava errado. Ex: maligno classificado como benigno. Custo alto!

Métrica	Fórmula	Quando Usar	Intervalo
Acurácia	$(VP + VN) / \text{Total}$	Classes balanceadas	0 – 1
Precision	$VP / (VP + FP)$	Minimizar alarmes falsos	0 – 1
Recall (Sensibilidade)	$VP / (VP + FN)$	Minimizar falsos negativos (diagnóstico)	0 – 1
F1-Score	$2 \times (P \times R) / (P + R)$	Classes desbalanceadas	0 – 1

Gerando as Métricas com Python

```
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, ConfusionMatrixDisplay)

# — ACURÁCIA —
print(f'Acurácia: {accuracy_score(y_teste, y_previsto):.4f}')

# — RELATÓRIO COMPLETO (precision, recall, F1 por classe)
print(classification_report(y_teste, y_previsto,
                             target_names=['Classe 0', 'Classe 1']))

# — VISUALIZAR MATRIZ DE CONFUSÃO —
cm = confusion_matrix(y_teste, y_previsto)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=['Neg.', 'Pos.'])
disp.plot(cmap='RdPu', colorbar=False)
plt.title('Matriz de Confusão'); plt.tight_layout(); plt.show()
```



Precision vs. Recall — Quando priorizar cada um?

Alta Precision → menos alarmes falsos. Prefira quando um FP tem custo alto (ex: bloquear transação legítima).

Alto Recall → menos casos perdidos. Prefira quando um FN tem custo alto (ex: não detectar câncer).

F1-Score equilibra os dois — use como métrica principal em datasets desbalanceados.

7.4 — K-Nearest Neighbors (KNN)

KNN é um dos algoritmos mais intuitivos do Machine Learning. Ele não cria um modelo durante o treino — apenas memoriza os dados. Na hora de classificar um novo ponto, calcula a distância até os K vizinhos mais próximos e escolhe a classe mais frequente entre eles.

Valor de K	Comportamento
K = 1 (pequeno)	Overfitting: muito sensível a ruído e outliers. Fronteiras irregulares.
K grande	Underfitting: generaliza demais, ignora padrões locais.
K ótimo	Encontrado via cross-validation. Balanceia variância e viés.

⚠ KNN EXIGE Normalização

KNN mede distâncias. Sem normalização, features com valores grandes (ex: renda em R\$) dominam sobre features pequenas (ex: número de filhos).

Sempre use StandardScaler ou MinMaxScaler antes de treinar e testar.

CRÍTICO: aplique fit_transform() no treino e apenas transform() no teste — nunca o contrário.

Script Genérico — KNN Classificação

```
# — 0. IMPORTAÇÕES —
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# — 1. CARREGAR E CONFIGURAR —
df = pd.read_csv('seu_dataset.csv')
TARGET = 'nome_da_coluna_target'
FEATURES = [c for c in df.columns if c != TARGET]
X = df[FEATURES]
y = df[TARGET]

# — 2. DIVISÃO TREINO / TESTE —
# stratify=y preserva a proporção das classes no treino e teste
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y)

# — 3. NORMALIZAÇÃO —
scaler = StandardScaler()
X_treino_scaled = scaler.fit_transform(X_treino) # fit + transform no TREINO
X_teste_scaled = scaler.transform(X_teste)      # apenas transform no TESTE

# — 4. ENCONTRAR MELHOR K VIA CROSS-VALIDATION —
erros = []
for k in range(1, 26):
    modelo_k = KNeighborsClassifier(n_neighbors=k)
    acuracias = cross_val_score(modelo_k, X_treino_scaled, y_treino,
                                cv=5, scoring='accuracy')
    erros.append(1 - acuracias.mean())

melhor_k = range(1, 26)[np.argmin(erros)]
print(f'Melhor K: {melhor_k} | Acurácia cross-val: {(1-min(erros))*100:.2f}%')
```

```
# plotar curva de erro x K
plt.plot(range(1, 26), erros, marker='o', color='#43054E')
plt.axvline(x=melhor_k, color='#BF0087', linestyle='--',
            label=f'Melhor K = {melhor_k}')
plt.title('Taxa de Erro x Valor de K')
plt.xlabel('K'); plt.ylabel('Erro')
plt.legend(); plt.tight_layout(); plt.show()

# — 5. TREINAR MODELO FINAL —
modelo = KNeighborsClassifier(n_neighbors=melhor_k)
modelo.fit(X_treino_scaled, y_treino)
y_previsto = modelo.predict(X_teste_scaled)

# — 6. AVALIAR —
print(classification_report(y_teste, y_previsto))
cm = confusion_matrix(y_teste, y_previsto)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='RdPu', colorbar=False)
plt.tight_layout(); plt.show()

# verificar overfitting
acc_treino = modelo.score(X_treino_scaled, y_treino)
acc_teste = modelo.score(X_teste_scaled, y_teste)
print(f'Treino: {acc_treino:.4f} | Teste: {acc_teste:.4f} | Gap: {abs(acc_treino-
acc_teste):.4f}')
```

Vantagens	Limitações
Fácil de entender e interpretar	Lento para datasets grandes (calcula distância para todos os pontos)
Não exige treinamento explícito	Exige normalização obrigatória dos dados
Funciona com múltiplas classes	Consome muita memória
Captura fronteiras não-lineares	Sofre com alta dimensionalidade (curse of dimensionality)
Pode ser usado para regressão	Sensível a features irrelevantes e a outliers

7.5 — Árvore de Decisão

Uma Árvore de Decisão aprende regras do tipo if-else para separar os dados em grupos cada vez mais homogêneos. Cada divisão busca reduzir a "impureza" dos grupos usando métricas como índice Gini ou Entropia.

Elemento	Descrição
Nó Raiz	Primeira e mais importante divisão — feature com maior poder de separação.
Nós Internos	Decisões intermediárias — cada pergunta divide o espaço de dados.
Folhas	Classes finais — nós sem filhos que atribuem um rótulo.
Gini (critério)	Mede impureza: 0 = puro (uma classe), 0,5 = máxima mistura (binário).
Entropia (critério)	Alternativa ao Gini. Mede incerteza. Resultados similares na prática.

max_depth	Efeito
None (sem limite)	Árvore cresce até separar todos os pontos → overfitting severo.
3 a 5 (recomendado)	Balanceia interpretabilidade e generalização.
1 ou 2 (raso)	Underfitting — regras muito simples, baixo poder preditivo.

✓ Por que Árvores são tão populares?

São altamente interpretáveis: é possível ler e explicar a regra ao cliente.

Não precisam de normalização dos dados.

Funcionam com features numéricas e categóricas.

São a base do Random Forest e do XGBoost — dois dos algoritmos mais poderosos.

Script Genérico — Árvore de Decisão

```
# — 0. IMPORTAÇÕES —————
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# — 1. CARREGAR E CONFIGURAR —————
df = pd.read_csv('seu_dataset.csv')
TARGET = 'nome_da_coluna_target'
FEATURES = [c for c in df.columns if c != TARGET]
X = df[FEATURES]
y = df[TARGET]

# — 2. DIVISÃO TREINO / TESTE —————
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y)

# — 3. ENCONTRAR MELHOR max_depth VIA CROSS-VALIDATION —
acuracias_cv = []
profundidades = range(1, 16)
for prof in profundidades:
    arvore_cv = DecisionTreeClassifier(max_depth=prof, random_state=42)
    scores = cross_val_score(arvore_cv, X_treino, y_treino, cv=5)
    acuracias_cv.append(scores.mean())

melhor_prof = list(profundidades)[np.argmax(acuracias_cv)]
print(f'Melhor profundidade: {melhor_prof}')

# plotar curva de acurácia x profundidade
plt.plot(profundidades, acuracias_cv, marker='o', color='#43054E')
plt.axvline(x=melhor_prof, color='#BF0087', linestyle='--',
            label=f'Melhor depth = {melhor_prof}')
plt.title('Acurácia × Profundidade (cross-val cv=5)')
plt.xlabel('max_depth'); plt.ylabel('Acurácia'); plt.legend()
plt.tight_layout(); plt.show()

# — 4. TREINAR MODELO FINAL —————
arvore = DecisionTreeClassifier(
    max_depth=melhor_prof,
```

```

        criterion='gini',
        random_state=42
    )
    arvore.fit(X_treino, y_treino)
    y_previsto = arvore.predict(X_teste)

# — 5. AVALIAR —
print(classification_report(y_teste, y_previsto))
cm = confusion_matrix(y_teste, y_previsto)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='RdPu', colorbar=False)
plt.tight_layout(); plt.show()

# — 6. VISUALIZAR A ÁRVORE —
plt.figure(figsize=(22, 10))
plot_tree(
    arvore,
    feature_names=FEATURES,
    class_names=[str(c) for c in arvore.classes_],
    filled=True, rounded=True, fontsize=10
)
plt.title('Árvore de Decisão', fontsize=14, fontweight='bold')
plt.tight_layout(); plt.show()

# — 7. IMPORTÂNCIA DAS FEATURES —
# features no topo da árvore (perto da raiz) têm maior importância
importancias = pd.Series(arvore.feature_importances_, index=FEATURES)
importancias.sort_values().plot(kind='barh', color='#43054E')
plt.title('Importância das Features'); plt.tight_layout(); plt.show()

```

Vantagens	Limitações
Altamente interpretável — explica decisões	Propenso a overfitting sem max_depth
Não precisa de normalização	Instável a pequenas mudanças nos dados
Funciona com dados numéricos e categóricos	Fronteiras de decisão sempre ortogonais
Captura interações entre variáveis	Menos preciso isoladamente que métodos ensemble
Base do Random Forest e XGBoost	Pode criar regras muito complexas

7.6 — Naive Bayes

Classificador probabilístico baseado no Teorema de Bayes. Calcula a probabilidade de cada classe dado um conjunto de features e escolhe a mais provável. O termo "naive" (ingênuo) vem da suposição de que todas as features são independentes entre si — uma hipótese raramente verdadeira, mas que funciona surpreendentemente bem em muitos cenários.

```

# — TEOREMA DE BAYES APLICADO —
# P(classe | dados) ∝ P(dados | classe) × P(classe)

from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB

# GaussianNB — features contínuas (assume distribuição normal)
modelo = GaussianNB()

# MultinomialNB — contagem de palavras (texto/NLP)

```

```
# modelo = MultinomialNB()

# BernoulliNB - features binárias (presença/ausência)
# modelo = BernoulliNB()

modelo.fit(X_treino, y_treino)
y_previsto = modelo.predict(X_teste)

# probabilidades por classe - saída contínua entre 0 e 1
probs = modelo.predict_proba(X_teste)
print(f'Probabilidades: {probs[:3]}')
```

Variante	Quando Usar
GaussianNB	Features contínuas com distribuição aproximadamente normal. Ex: altura, peso, temperatura.
MultinomialNB	Contagem de eventos. Muito usado em NLP para classificação de textos.
BernoulliNB	Features binárias (0 ou 1). Ex: presença/ausência de palavras.

7.7 — Regressão Logística

Apesar do nome, é um algoritmo de classificação. Modela a probabilidade de uma amostra pertencer a uma classe aplicando uma combinação linear das features e passando o resultado pela função sigmoide — transformando qualquer valor em uma probabilidade entre 0 e 1.

Funcionamento passo a passo

1. Calcula uma combinação linear das variáveis: $z = w_0 + w_1x_1 + w_2x_2 + \dots$
2. Aplica a função sigmoide: $P = 1 / (1 + e^{-z})$
3. Gera uma probabilidade entre 0 e 1 para a classe positiva.
4. Aplica um limiar (padrão: 0,5) para decidir a classe final.

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

# — NORMALIZAÇÃO RECOMENDADA PARA CONVERGÊNCIA —
scaler = StandardScaler()
X_treino_scaled = scaler.fit_transform(X_treino)
X_teste_scaled = scaler.transform(X_teste)

# — TREINAR —
modelo = LogisticRegression(random_state=42, max_iter=1000)
modelo.fit(X_treino_scaled, y_treino)
y_previsto = modelo.predict(X_teste_scaled)

# — PROBABILIDADES (útil para ajuste de threshold) —
probs = modelo.predict_proba(X_teste_scaled)[:, 1] # P(classe=1)

# — COEFICIENTES - INTERPRETABILIDADE —
# coeficiente positivo → aumenta probabilidade da classe 1
# coeficiente negativo → diminui probabilidade da classe 1
coef = pd.Series(modelo.coef_[0], index=FEATURES).sort_values()
coef.plot(kind='barh',
          color=['#BF0087' if v > 0 else '#43054E' for v in coef])
```

```
plt.title('Coeficientes da Regressão Logística')
plt.tight_layout(); plt.show()
```

7.8 — Comparativo dos Algoritmos

Critério	KNN	Árvore de Decisão	Naive Bayes	Reg. Logística
Interpretabilidade	Baixa	Alta ✓	Média	Alta ✓
Precisa Normalizar?	Sim (obrigatório)	Não	Não (GaussianNB)	Recomendado
Velocidade (treino)	Nenhuma	Rápida	Muito rápida ✓	Rápida
Velocidade (predict)	Lenta ⚠	Rápida	Rápida	Rápida
Datasets grandes	Ruim ⚠	Bom	Excelente ✓	Bom
Dados categóricos	Com encoding	Nativo ✓	Com encoding	Com encoding
Risco de overfitting	K pequeno	Sem max_depth	Baixo	Baixo
Melhor uso	Poucos dados, rec. sistemas	Crédito, triagem, regras	Texto, spam, NLP	Churn, risco, baseline

7.9 — Checklist CRISP-DM para Classificação

F1 Entendimento do Negócio

- ☐ O problema está definido: "prever se [sujeito] vai [classe A] ou [classe B]"
- ☐ A variável-alvo (target) foi identificada e é categórica
- ☐ Tipo definido: binária / multiclasse / multilabel
- ☐ Critérios de sucesso estabelecidos (ex: F1 > 0.85, Recall > 0.90)

F2 Entendimento dos Dados

- ☐ df.shape, df.dtypes, df.describe(), df.isnull().sum() executados
- ☐ Distribuição do target analisada — classes balanceadas?
- ☐ Correlação das features com o target visualizada
- ☐ Histogramas e boxplots por classe gerados

F3 Preparação dos Dados

- ☐ Valores ausentes tratados (mediana para contínuos, moda para categóricos)
- ☐ Encoding aplicado: Label para ordinais, One-Hot para nominais
- ☐ Normalização: StandardScaler aplicado (obrigatório para KNN e Reg. Logística)
- ☐ Divisão treino/teste com stratify=y para preservar proporção das classes

F4 Modelagem

- ☐ Algoritmo escolhido com justificativa (KNN, Árvore, Naive Bayes, Log. Reg.)
- ☐ Hiperparâmetros testados via `cross_val_score` (K para KNN, `max_depth` para Árvore)
- ☐ Modelo final treinado com melhor hiperparâmetro
- ☐ Gap treino/teste verificado (< 5% = OK, > 15% = overfitting)

F5 Avaliação

- ☐ Matriz de Confusão gerada e interpretada (VP, VN, FP, FN)
- ☐ `classification_report` impresso (precision, recall, F1 por classe)
- ☐ Métrica prioritária definida (acurácia / F1 / recall) conforme contexto de negócio
- ☐ Resultado atinge o critério de sucesso definido na F1?

Lembrete Final — Processo Cíclico

Se o desempenho na Avaliação não for suficiente → volte à Preparação para engenharia de features.
Se ainda não for suficiente → volte à Modelagem e teste outros algoritmos ou ajuste hiperparâmetros.
O CRISP-DM não é linear. Iterar é a norma, não a exceção.

PIPELINE COMPLETA — SCRIPT DO INÍCIO AO FIM

Abaixo está o script completo consolidado, do carregamento ao modelo treinado. Adapte os nomes de colunas conforme seu dataset.

```
# =====
# PIPELINE COMPLETA CRISP-DM — REGRESSÃO LINEAR
# Substitua os nomes genéricos pelos do seu projeto
# =====

# FASE 0 — SETUP
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import pickle
%matplotlib inline
plt.rcParams['figure.figsize'] = (12, 5)
sns.set_theme(style='whitegrid')

# FASE 2 — CARREGAR E EXPLORAR
df = pd.read_csv('seu_arquivo.csv')
print(f"Shape: {df.shape}")
print(df.isnull().sum())

# FASE 3 — PREPARAR DADOS
df_limpo = df.copy()
df_limpo['coluna_nulos1'] =
df_limpo['coluna_nulos1'].fillna(df_limpo['coluna_nulos1'].median())
df_limpo['coluna_nulos2'] =
df_limpo['coluna_nulos2'].fillna(df_limpo['coluna_nulos2'].median())
df_limpo = df_limpo[df_limpo['coluna_alvo'] <= LIMITE_MAX] # remover outliers
df_limpo['feature_nova'] = df_limpo['colA'] / df_limpo['colB'] # nova feature

# FASE 4 — MODELAGEM
FEATURES = ['feat1', 'feat2', 'feat3']
TARGET = 'coluna_alvo'
X = df_limpo[FEATURES]
y = df_limpo[TARGET]
X_treino, X_teste, y_treino, y_teste = train_test_split(X, y, test_size=0.2,
random_state=42)
modelo = LinearRegression()
modelo.fit(X_treino, y_treino)
y_pred_treino = modelo.predict(X_treino)
y_pred_teste = modelo.predict(X_teste)

# FASE 5 — AVALIAÇÃO
rmse_tr = np.sqrt(mean_squared_error(y_treino, y_pred_treino))
rmse_te = np.sqrt(mean_squared_error(y_teste, y_pred_teste))
r2_tr = r2_score(y_treino, y_pred_treino)
r2_te = r2_score(y_teste, y_pred_teste)
print(f"RMSE Treino: {rmse_tr:.2f} | Teste: {rmse_te:.2f}")
print(f"R² Treino: {r2_tr:.4f} | Teste: {r2_te:.4f}")

# FASE 6 — SALVAR
with open('modelo_final.pkl', 'wb') as f:
    pickle.dump(modelo, f)
print('Pipeline completa — modelo salvo!')
```


APÊNDICE A — COMANDOS PANDAS ESSENCIAIS

Referência rápida dos comandos Pandas mais utilizados em Ciência de Dados.

A.1 — Carregamento e Inspeção

Comando	O que faz	Exemplo
<code>pd.read_csv('arq.csv')</code>	Lê arquivo CSV como DataFrame	<code>df = pd.read_csv('dados.csv')</code>
<code>pd.read_excel('arq.xlsx')</code>	Lê planilha Excel	<code>df = pd.read_excel('plan.xlsx')</code>
<code>df.head(n)</code>	Mostra as primeiras n linhas (padrão: 5)	<code>df.head(10)</code>
<code>df.tail(n)</code>	Mostra as últimas n linhas	<code>df.tail(5)</code>
<code>df.shape</code>	Retorna (linhas, colunas)	<code>print(df.shape)</code>
<code>df.columns</code>	Lista os nomes das colunas	<code>print(df.columns.tolist())</code>
<code>df.dtypes</code>	Tipo de dado de cada coluna	<code>print(df.dtypes)</code>
<code>df.info()</code>	Resumo: tipos, nulos, memória	<code>df.info()</code>
<code>df.describe()</code>	Estatísticas descritivas (count, mean, std, min, max...)	<code>df.describe().T</code>
<code>df.nunique()</code>	Nº de valores únicos por coluna	<code>df.nunique()</code>

A.2 — Seleção e Filtragem

Comando	O que faz	Exemplo
<code>df['col']</code>	Seleciona uma coluna (retorna Series)	<code>df['preco_mil']</code>
<code>df[['c1','c2']]</code>	Seleciona múltiplas colunas (retorna DataFrame)	<code>df[['area_m2','preco_mil']]</code>
<code>df.loc[linha, col]</code>	Seleciona por rótulo (label)	<code>df.loc[0:5, ['tipo','preco']]</code>
<code>df.iloc[linha, col]</code>	Seleciona por posição inteira	<code>df.iloc[0:3, 0:4]</code>
<code>df[df['col'] > x]</code>	Filtra linhas por condição	<code>df[df['preco_mil'] > 500]</code>
<code>df[(c1) & (c2)]</code>	Filtra com múltiplas condições (E)	<code>df[(df['tipo']=='Casa') & (df['area']>100)]</code>
<code>df[(c1) (c2)]</code>	Filtra com múltiplas condições (OU)	<code>df[(df['tipo']=='Casa') (df['tipo']=='Apt')]</code>
<code>df.query('expr')</code>	Filtra com expressão textual	<code>df.query('preco_mil > 1000 and bairro=="Lago Sul"')</code>
<code>df['col'].isin(lista)</code>	Verifica se valor está na lista	<code>df[df['tipo'].isin(['Casa','Cobertura'])]</code>

A.3 — Manipulação e Transformação

Comando	O que faz	Exemplo
<code>df['col'].fillna(v)</code>	Preenche nulos com valor v	<code>df['vagas'].fillna(df['vagas'].median())</code>
<code>df.dropna()</code>	Remove linhas com qualquer nulo	<code>df.dropna(subset=['col_importante'])</code>
<code>df.drop('col', axis=1)</code>	Remove uma coluna	<code>df.drop('col_desnecessaria', axis=1)</code>
<code>df.rename(columns={old:new})</code>	Renomeia colunas	<code>df.rename(columns={'antigo':'novo'})</code>
<code>df.sort_values(col)</code>	Ordena pelo valor de uma coluna	<code>df.sort_values('preco_mil', ascending=False)</code>
<code>df.copy()</code>	Cria cópia independente	<code>df_limpo = df.copy()</code>
<code>df.reset_index(drop=True)</code>	Reinicia o índice	<code>df = df.reset_index(drop=True)</code>
<code>df['col'] = expressao</code>	Cria ou sobrescreve coluna	<code>df['preco_m2'] = df['preco_mil']/df['area_m2']</code>
<code>pd.get_dummies(df, columns=['c'])</code>	One-hot encoding de colunas categóricas	<code>pd.get_dummies(df, columns=['tipo'])</code>

A.4 — Agregação e Agrupamento

Comando	O que faz	Exemplo
<code>df.groupby('col').mean()</code>	Agrupa e calcula média	<code>df.groupby('bairro')['preco_mil'].mean()</code>
<code>df.groupby('col').median()</code>	Agrupa e calcula mediana	<code>df.groupby('tipo')['area_m2'].median()</code>
<code>df.groupby('col').agg({c:'func'})</code>	Agrupa com funções diferentes por coluna	<code>df.groupby('bairro').agg({'preco':'mean','area':'max'})</code>
<code>df['col'].value_counts()</code>	Conta ocorrências de cada valor único	<code>df['tipo'].value_counts()</code>
<code>df['col'].value_counts(normalize=True)</code>	Proporção de cada valor único	<code>df['bairro'].value_counts(normalize=True)</code>
<code>df.pivot_table(...)</code>	Tabela dinâmica (como no Excel)	<code>df.pivot_table(values='preco', index='bairro', aggfunc='mean')</code>

A.5 — Nulos e Tipos

Comando	O que faz	Exemplo
<code>df.isnull().sum()</code>	Conta nulos por coluna	<code>print(df.isnull().sum())</code>

<code>df.isna().sum()</code>	Igual ao <code>isnull()</code> (sinônimo)	<code>print(df.isna().sum())</code>
<code>df.notnull()</code>	True onde valor NÃO é nulo	<code>df[df['col'].notnull()]</code>
<code>df['col'].astype(tipo)</code>	Converte tipo da coluna	<code>df['ano'].astype(int)</code>
<code>df.select_dtypes(include='number')</code>	Seleciona apenas colunas numéricas	<code>num = df.select_dtypes(include='number')</code>
<code>df['col'].str.lower()</code>	Transforma texto para minúsculas	<code>df['nome'].str.lower()</code>
<code>df['col'].str.strip()</code>	Remove espaços extras	<code>df['bairro'].str.strip()</code>

APÊNDICE B — GRÁFICOS MATPLOTLIB (plt) E SEABORN (sns)

Referência dos gráficos mais usados em Ciência de Dados, com código genérico e quando utilizar cada um.

B.1 — Estrutura Base de um Gráfico

```
# — ANATOMIA BÁSICA (matplotlib) —————
# fig = a "folha" onde o gráfico é desenhado
# ax = os eixos onde os dados são plotados
fig, ax = plt.subplots(figsize=(10, 6))

# (seu código de gráfico aqui)

ax.set_title('Título do Gráfico', fontsize=14, fontweight='bold')
ax.set_xlabel('Nome do Eixo X', fontsize=12)
ax.set_ylabel('Nome do Eixo Y', fontsize=12)
ax.legend() # mostra a legenda
plt.tight_layout() # ajusta margens automaticamente
plt.show() # exibe o gráfico

# — MÚLTIPLOS GRÁFICOS (subplots) —————
# cria 1 linha e 2 colunas de gráficos lado a lado
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
# axes[0] = gráfico da esquerda
# axes[1] = gráfico da direita
```

B.2 — Catálogo de Gráficos Matplotlib (plt)

HISTOGRAMA — Distribuição de uma variável numérica

```
# Use para: ver a distribuição de frequência de uma variável numérica
# Responde: os valores estão concentrados onde? há assimetria?
plt.hist(df['coluna_numerica'], bins=30,
         color='#2E75B6', edgecolor='white', alpha=0.85)
plt.axvline(df['coluna_numerica'].mean(),
            color='red', linestyle='--', linewidth=2, label='Média')
plt.axvline(df['coluna_numerica'].median(),
            color='green', linestyle=':', linewidth=2, label='Mediana')
plt.title('Distribuição de coluna_numerica')
plt.xlabel('Valor') # nome do eixo X
plt.ylabel('Frequência') # nome do eixo Y
plt.legend()
plt.show()
```

SCATTER (Dispersão) — Relação entre duas variáveis numéricas

```
# Use para: verificar correlação visual entre duas variáveis
# Responde: quando X aumenta, Y também aumenta?
plt.scatter(df['variavel_x'], df['variavel_y'],
            alpha=0.5, # transparência (0=invisível, 1=opaco)
```

```
        color='#1B3A6B',
        s=40,           # tamanho dos pontos
        edgecolors='white')
plt.title('Variável X vs Variável Y')
plt.xlabel('Variável X')
plt.ylabel('Variável Y')
plt.show()
```

BAR CHART (Barras) — Comparação de categorias

```
# Use para: comparar valores entre categorias
# Responde: qual categoria tem o maior/menor valor?
contagem = df['coluna_categorica'].value_counts()

plt.bar(contagem.index, contagem.values, color='#2E75B6', edgecolor='white')
plt.title('Contagem por Categoria')
plt.xlabel('Categoria')
plt.ylabel('Contagem')
plt.xticks(rotation=45) # rotaciona os rótulos do eixo X
plt.tight_layout()
plt.show()
```

B.3 — Catálogo de Gráficos Seaborn (sns)

O Seaborn é construído sobre o matplotlib e oferece gráficos estatísticos com visual mais elaborado e integração direta com DataFrames.

sns.histplot — Histograma com curva de densidade (KDE)

```
# Use para: ver distribuição + comparar por categoria (hue)
sns.histplot(data=df, x='coluna_numerica',
             hue='coluna_categorica', # separar por cor
             kde=True,               # curva de densidade suave
             element='step')         # estilo: step, bars, poly
plt.title('Distribuição por Categoria')
plt.show()
```

sns.boxplot — Resumo estatístico e outliers

```
# Use para: comparar distribuições entre grupos
# Mostra: Q1, mediana, Q3, e pontos fora = outliers
sns.boxplot(data=df, x='coluna_categorica', y='coluna_numerica',
            order=['Cat_A', 'Cat_B', 'Cat_C'], # ordem das categorias
            palette='Set2')                   # paleta de cores
plt.title('Distribuição de Valores por Grupo')
plt.show()
```

sns.scatterplot — Dispersão com categorias por cor

```
# Use para: scatter com separação visual por categoria
sns.scatterplot(data=df, x='variavel_x', y='variavel_y',
                hue='coluna_categorica', # cor por categoria
```

```

        alpha=0.6, s=60)
plt.title('Variável X vs Y por Categoria')
plt.show()

```

sns.heatmap — Mapa de calor (correlação / nulos)

```

# — Heatmap de Correlação —————
numericas = df.select_dtypes(include='number')
sns.heatmap(numericas.corr(), annot=True, fmt='.2f',
            cmap='coolwarm', # coolwarm, Blues, RdYlGn...
            center=0, # centraliza a escala de cor no 0
            linewidths=0.5, # linhas entre células
            annot_kws={'size': 9})
plt.title('Matriz de Correlação de Pearson')
plt.show()

# — Heatmap de Nulos —————
sns.heatmap(df.isnull(), cbar=False, cmap='viridis', yticklabels=False)
plt.title('Mapa de Valores Ausentes')
plt.show()

```

sns.lmplot — Regressão visual por grupo

```

# Use para: visualizar a linha de regressão por categoria
g = sns.lmplot(data=df, x='variavel_x', y='variavel_y',
              hue='coluna_categorica', # linha por categoria
              height=5, aspect=2,
              scatter_kws={'alpha': 0.35, 's': 35},
              line_kws={'linewidth': 2.5}, ci=95)
g.figure.suptitle('Regressão Visual por Grupo', y=1.02)
g.set_axis_labels('Eixo X', 'Eixo Y')
plt.tight_layout()
plt.show()

```

sns.violinplot — Distribuição completa por grupo

```

# Use para: combinar boxplot + distribuição de densidade
sns.violinplot(data=df, x='coluna_categorica', y='coluna_numerica',
              inner='box') # inner: box, quartile, point, stick
plt.title('Distribuição por Grupo (Violin)')
plt.show()

```

sns.barplot — Média com intervalo de confiança

```

# Use para: comparar médias entre grupos, com erro padrão
sns.barplot(data=df, x='coluna_categorica', y='coluna_numerica',
            estimator='mean', # pode ser median, sum...
            errorbar='ci', # mostra intervalo de confiança
            palette='Blues_d')
plt.title('Média por Categoria (com IC 95%)')
plt.show()

```


B.4 — Tabela Resumo: Qual Gráfico Usar?

Pergunta que você quer responder	Gráfico Recomendado	Biblioteca
Como os valores de uma variável se distribuem?	histplot, hist	sns, plt
Existem outliers em uma coluna?	boxplot, violinplot	sns
Qual a relação entre X e Y?	scatter, scatterplot, lmpplot	plt, sns
Como as categorias se comparam?	barplot, bar	sns, plt
Qual a correlação entre todas as variáveis?	heatmap (corr)	sns
Onde estão os valores ausentes?	heatmap (isnull)	sns
Qual a distribuição X por categoria?	boxplot, violinplot, histplot com hue	sns
Como a regressão se comporta por grupo?	lmpplot	sns

B.5 — Dicas de Personalização Visual

```
# — PALETAS DE CORES SEABORN —
# Qualitativas (para categorias): 'Set1','Set2','Set3','tab10','Paired'
# Sequenciais (para valores):      'Blues','Greens','Oranges','viridis'
# Divergentes (para correlação):  'coolwarm','RdBu','RdYlGn'

# — TEMAS SEABORN —
# whitegrid: fundo branco com grade cinza (mais comum)
# darkgrid:  fundo cinza com grade
# white:     fundo branco sem grade
# ticks:     apenas marcas nos eixos
sns.set_theme(style='whitegrid', palette='Set2')

# — FORMATAR EIXO Y COMO MOEDA (R$) —
import matplotlib.ticker as mticker
ax.yaxis.set_major_formatter(mticker.FuncFormatter(
    lambda x, _: f'R$ {x:,.0f}'))

# — ADICIONAR RÓTULOS NAS BARRAS —
for bar in ax.patches:
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.5,
        f'{bar.get_height():.1f}',
        ha="center", va="bottom", fontsize=9
    )
```

APÊNDICE C — ERROS COMUNS E SOLUÇÕES

Mensagem de Erro	Causa Provável	Solução
FileNotFoundError: nome.csv	Arquivo não está no caminho informado	Verifique o caminho. No Colab use files.upload()
KeyError: nome_coluna	Nome de coluna errado (maiúsculas, espaços)	Use df.columns para ver os nomes exatos
ValueError: could not convert string to float	Coluna tem texto onde se espera número	Use pd.to_numeric(df['col'], errors='coerce')
Cannot use mean with non-numeric data	Tentou calcular média em coluna de texto	Selecione apenas colunas numéricas: df.select_dtypes('number')
Input contains NaN	Há nulos nas features antes do train/test split	Execute tratamento de nulos antes de criar X e y
R ² negativo	Modelo pior que prever a média — overfitting severo ou bug	Verifique se está usando y_pred do teste com y_teste
RMSE muito alto (>50% da média)	Features erradas, nulos não tratados, outliers no target	Revisar tratamento de dados e seleção de features

APÊNDICE D — MÉTRICAS DE AVALIAÇÃO DE MODELOS

Referência completa das principais métricas usadas para avaliar modelos de Machine Learning, organizadas por tipo de problema. Use esta tabela para escolher e justificar as métricas do seu projeto.

D.1 — Métricas de Regressão

Usadas quando o target é um valor numérico contínuo (preço, temperatura, salário, etc.).

Métrica	Fórmula simplificada	Interpretação	Vantagem	Limitação	Sklearn
R ² Coef. Determinação	$r^2 = 1 - (SS_{res} / SS_{tot})$	0 a 1: proporção da variância explicada pelo modelo. Negativo = pior que a média.	Adimensional — compara modelos independentemente da escala	Sempre sobe ao adicionar features — use com cautela	<code>r2_score(y, ŷ)</code>
RMSE Raiz Erro Quad. Médio	$\sqrt{\text{médio}((y - \hat{y})^2)}$	Erro médio na unidade do target. RMSE = 50k → erra em média R\$ 50 mil.	Penaliza erros grandes — útil quando erros graves são custosos	Sensível a outliers	<code>np.sqrt(mean_squared_error(y, ŷ))</code>
MAE Erro Médio Absoluto	$\text{médio}(y - \hat{y})$	Erro médio na unidade do target. Trata todos os erros da mesma forma.	Robusto a outliers — interpretação direta	Não penaliza erros grandes	<code>mean_absolute_error(y, ŷ)</code>
MAPE Erro % Médio Absoluto	$\text{médio}(y - \hat{y} / y) \times 100$	Erro médio em percentual. MAPE = 8% → modelo erra em média 8% do valor real.	Intuitivo para negócios — independente da escala do target	Instável quando target tem valores próximos de zero	<code>mean_absolute_percentage_error(y, ŷ)</code>
MSE Erro Quad. Médio	$\text{médio}((y - \hat{y})^2)$	Versão sem raiz do RMSE. Menos interpretável mas útil para otimização matemática.	Base para o RMSE e para funções de perda em deep learning	Unidade ao quadrado — difícil de interpretar diretamente	<code>mean_squared_error(y, ŷ)</code>

D.2 — Métricas de Classificação

Usadas quando o target é uma categoria (sim/não, tipo A/B/C, spam/não spam, etc.). Serão aprofundadas no módulo de Classificação.

Métrica	Interpretação	Quando usar	Limitação	Sklearn
Acurácia	% de previsões corretas no total.	Classes balanceadas — quantidade similar de cada categoria.	Enganosa em dados desbalanceados: 95% "não" → acurácia alta sem aprender nada.	<code>accuracy_score(y, $\hat{y}$)</code>
Precisão	Dos que o modelo disse "sim", quantos eram realmente "sim"?	Quando falsos positivos são custosos (ex: spam, fraude).	Alta precisão pode significar baixo recall.	<code>precision_score(y, $\hat{y}$)</code>
Recall (Sensibilidade)	Dos que eram "sim", quantos o modelo identificou?	Quando falsos negativos são custosos (ex: diagnóstico médico).	Alto recall pode significar baixa precisão.	<code>recall_score(y, $\hat{y}$)</code>
F1-Score	Média harmônica entre Precisão e Recall — equilibra os dois.	Dados desbalanceados ou quando precisão e recall importam igualmente.	Não considera verdadeiros negativos.	<code>f1_score(y, $\hat{y}$)</code>
AUC-ROC	Capacidade do modelo de separar classes. 0.5 = aleatório, 1.0 = perfeito.	Comparar modelos — independe do limiar de decisão.	Menos intuitivo para comunicar resultados a não-técnicos.	<code>roc_auc_score(y, $\hat{y}$)</code>
Matriz de Confusão	Tabela VP / FP / VN / FN — mostra exatamente onde o modelo acerta e erra.	Diagnóstico detalhado do comportamento do modelo.	Não resume em um único número.	<code>confusion_matrix(y, $\hat{y}$)</code>

D.3 — Qual Métrica Usar em Cada Situação?

Situação	Métricas recomendadas	Observação
Regressão — uso geral	R^2 , RMSE, MAE	Calcule os três e compare treino vs teste
Regressão — comunicar para negócios	MAPE, MAE	Percentual e valor absoluto são mais intuitivos que R^2
Regressão — dados com outliers	MAE, MAPE	RMSE distorcido por outliers — prefira MAE

Classificação — classes balanceadas	Acurácia, AUC-ROC	Acurácia é suficiente quando classes são equilibradas
Classificação — classes desbalanceadas	F1-Score, AUC-ROC	Acurácia engana — use F1 ou AUC-ROC
Classificação — erro grave é caro	Recall (médico, fraude)	Prefira não perder um caso real mesmo com mais falsos positivos
Classificação — alarme falso é caro	Precisão (spam, alertas)	Prefira errar menos ao dizer "sim" mesmo perdendo alguns casos
Comparar múltiplos modelos	R^2 ou AUC-ROC + RMSE/F1	Use AUC-ROC para classificação e R^2 +RMSE para regressão

Overfitting — sinal de alerta em qualquer métrica

Compare sempre a métrica no TREINO com a métrica no TESTE.

Regressão: RMSE teste > RMSE treino em mais de 20% → overfitting.

Classificação: Acurácia treino muito maior que Acurácia teste → overfitting.

Solução: reduzir features, aumentar dados ou aplicar regularização.

Na dúvida, qual métrica priorizar em regressão?

MAE é a mais segura quando você não conhece bem os dados — robusta a outliers e fácil de explicar para não-técnicos.

RMSE é melhor quando erros grandes são inaceitáveis — penaliza discrepâncias graves.

R^2 é indispensável para responder "o modelo é bom?" e comparar versões.

Regra de bolso: se $RMSE \approx MAE$ → erros homogêneos. Se $RMSE \gg MAE$ → há erros extremos — investigar.

Use os três juntos: R^2 para qualidade geral, MAE para erro típico, RMSE para detectar erros graves.

APÊNDICE E — SCRIPT DE DIAGNÓSTICO COMPLETO DE MÉTRICAS

Execute este script após treinar qualquer modelo de regressão para obter um diagnóstico completo e automático — métricas calculadas, comparação treino vs teste, detecção de overfitting e interpretação em linguagem natural.

Pré-requisitos

O modelo deve estar treinado e as variáveis `y_treino`, `y_teste`, `y_pred_treino` e `y_pred_teste` devem estar definidas.

Substitua `unidade_target` pela unidade real do seu problema — ex: "R\$", "kg", "°C", "unidades".

Métrica	Biblioteca	Importação
R ²	scikit-learn	<code>from sklearn.metrics import r2_score</code>
RMSE	scikit-learn + numpy	<code>from sklearn.metrics import mean_squared_error</code> <code>import numpy as np</code> <code># np.sqrt()</code>
MAE	scikit-learn	<code>from sklearn.metrics import mean_absolute_error</code>
MAPE	scikit-learn ≥ 0.24	<code>from sklearn.metrics import mean_absolute_percentage_error</code>
MSE	scikit-learn	<code>from sklearn.metrics import mean_squared_error</code>
Pandas	pandas	<code>import pandas as pd</code> # para montar a tabela de resultados

Versão mínima do scikit-learn

`mean_absolute_percentage_error` (MAPE) foi adicionado na versão 0.24 do scikit-learn (2021).

Para verificar a versão instalada: `import sklearn; print(sklearn.__version__)`

Para atualizar: `pip install --upgrade scikit-learn`

```
# =====
# APÊNDICE E — DIAGNÓSTICO COMPLETO DE MÉTRICAS
# =====
from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error,
    mean_absolute_percentage_error,
    r2_score
)
import numpy as np
import pandas as pd

# — CONFIGURAÇÃO —
# substitua pela unidade real do seu target
unidade_target = 'R$'

# — BLOCO 1: CALCULAR MÉTRICAS —
# treino
r2_tr = r2_score(y_treino, y_pred_treino)
rmse_tr = np.sqrt(mean_squared_error(y_treino, y_pred_treino))
mae_tr = mean_absolute_error(y_treino, y_pred_treino)
mape_tr = mean_absolute_percentage_error(y_treino, y_pred_treino) * 100
```

```

# teste
r2_te = r2_score(y_teste, y_pred_teste)
rmse_te = np.sqrt(mean_squared_error(y_teste, y_pred_teste))
mae_te = mean_absolute_error(y_teste, y_pred_teste)
mape_te = mean_absolute_percentage_error(y_teste, y_pred_teste) * 100

# — BLOCO 2: TABELA COMPARATIVA —————
resumo = pd.DataFrame({
    'Métrica': ['R²', 'RMSE', 'MAE', 'MAPE (%)'],
    'Treino': [round(r2_tr,4), round(rmse_tr,2),
               round(mae_tr,2), round(mape_tr,2)],
    'Teste': [round(r2_te,4), round(rmse_te,2),
              round(mae_te,2), round(mape_te,2)],
})

print('=' * 50)
print(' MÉTRICAS — TREINO vs TESTE ')
print('=' * 50)
print(resumo.to_string(index=False))
print('=' * 50)

# — BLOCO 3: DIAGNÓSTICO RMSE vs MAE —————
# ratio próximo de 1 → erros homogêneos
# ratio alto → há erros grandes/extremos
ratio = rmse_te / mae_te if mae_te > 0 else 1

print('\n— DIAGNÓSTICO RMSE vs MAE —')
if ratio > 2.0:
    print(f' Ratio RMSE/MAE = {ratio:.2f} → ATENÇÃO: erros
          grandes detectados — investigar outliers nas previsões')
elif ratio > 1.5:
    print(f' Ratio RMSE/MAE = {ratio:.2f} → Alguns erros
          relevantes presentes — monitorar')
else:
    print(f' Ratio RMSE/MAE = {ratio:.2f} → Erros homogêneos
          — modelo estável')

# — BLOCO 4: DIAGNÓSTICO DE OVERFITTING —————
# compara cada métrica entre treino e teste
dif_r2 = r2_tr - r2_te # positivo = treino melhor
dif_rmse = (rmse_te - rmse_tr) / rmse_tr * 100 # % degradação
dif_mae = (mae_te - mae_tr) / mae_tr * 100 # % degradação

print('\n— DIAGNÓSTICO DE OVERFITTING —')

def diagnostico(dif_pct, metrica):
    if dif_pct < 10:
        return f' {metrica}: Bom ajuste (+{dif_pct:.1f}%)'
    elif dif_pct < 20:
        return f' {metrica}: Leve overfitting (+{dif_pct:.1f}%) — aceitável'
    else:
        return f' {metrica}: OVERFITTING (+{dif_pct:.1f}%) — revisar features'

print(diagnostico(dif_rmse, "RMSE"))
print(diagnostico(dif_mae, "MAE "))

```

```
# — BLOCO 5: INTERPRETAÇÃO EM LINGUAGEM NATURAL —  
print('\n— INTERPRETAÇÃO —')  
print(f" R² = {r2_te:.4f} → o modelo explica",  
      f"{r2_te*100:.1f}% da variação do target")  
print(f" MAE = {mae_te:.2f} → o modelo erra em média",  
      f"{unidade_target} {mae_te:,.2f} por previsão")  
print(f" RMSE= {rmse_te:.2f} → penalizando erros grandes,",  
      f"o desvio típico é {unidade_target} {rmse_te:,.2f}")  
print(f" MAPE= {mape_te:.2f}% → em percentual, o modelo",  
      f"erra em média {mape_te:.1f}% do valor real")
```

✓ Como interpretar a saída

R^2 próximo de 1 + $RMSE \approx MAE$ + diferença treino/teste < 10% = modelo excelente.

Se $MAPE < 10\%$ → ótimo. Entre 10–20% → aceitável. Acima de 20% → revisar o modelo.

Ratio $RMSE/MAE > 2$ é um alerta para investigar quais registros têm erros extremos.

Rode este script para cada versão do modelo (A, B, C do tópico 5.3.3) e compare os resultados.

GLOSSÁRIO TÉCNICO

Termo	Definição
DataFrame	Estrutura de dados tabular do Pandas — como uma planilha Excel programável
Series	Uma coluna de um DataFrame — lista unidimensional com índice
Feature / Variável independente	Coluna de entrada usada pelo modelo para aprender (ex: área_m2, quartos)
Target / Variável dependente	Coluna que o modelo deve prever (ex: preco_mil)
EDA	Exploratory Data Analysis — exploração visual e estatística dos dados
Encoding	Conversão de variável categórica (texto) em número para o modelo entender
One-hot Encoding	Cria colunas binárias (0 ou 1) para cada categoria — mais comum
Outlier	Valor atípico — muito diferente da maioria dos dados
IQR	Intervalo Interquartil (Q3 - Q1) — método para detectar outliers
train_test_split	Divisão automática em dados de treino (aprender) e teste (avaliar)
Overfitting	Modelo "decorou" o treino mas falha em dados novos (treino ótimo, teste ruim)
Underfitting	Modelo muito simples para capturar os padrões (treino ruim, teste ruim)
RMSE	Root Mean Squared Error — raiz do erro quadrático médio — mesma unidade do target
MAE	Mean Absolute Error — erro médio absoluto — robusto a outliers
R ²	Coefficiente de determinação — de 0 a 1 — quanto o modelo explica a variância
Correlação de Pearson	Mede a relação linear entre duas variáveis (de -1 a +1)
pickle	Biblioteca Python para serializar objetos — salvar modelos em arquivo
random_state	Semente aleatória — garante reprodutibilidade dos resultados
CRISP-DM	Metodologia padrão para projetos de Data Science em 6 fases iterativas